
More than One Way to Skin a Cat: Preserving Verified Modes in RLVR

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Correctness alone does not reveal how many successful solutions a model can
2 produce. We introduce MODEBENCH, five executable domains that distinguish
3 correct outputs for the same problem. Replaying verified outputs during Dr.GRPO
4 and MaxRL improves held-out correctness and sampled breadth across model
5 scales and on harder tasks. An inference-only comparison also finds concentrated
6 outputs in highly accurate hosted models. These results distinguish discovering a
7 successful behavior from keeping it available.

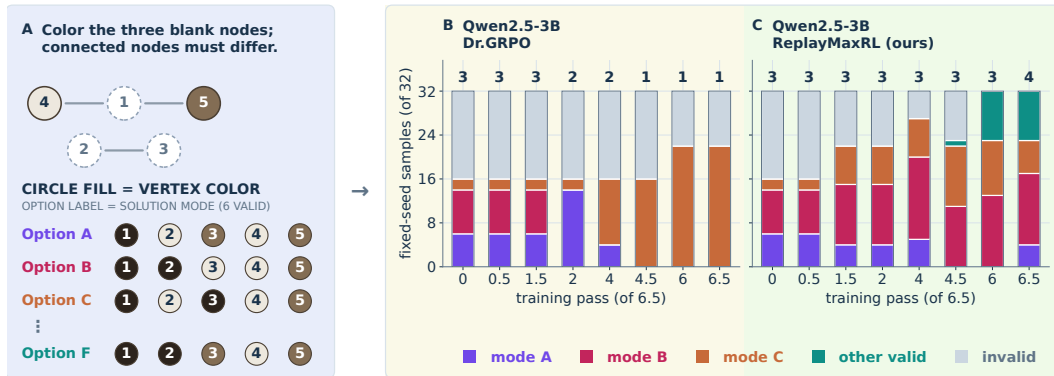


Figure 1: **Correctness can rise while alternatives disappear.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct from one mode; ReplayMaxRL produces 32/32 across four.

1 Preserving alternatives for mathematical agents

9 Mathematical agents use tools and verifiers to search and revise [Brown et al., 2024, Snell et al.,
10 2024], but can exploit alternative computations only if the model proposes them. Reinforcement
11 learning with verifiable rewards (RLVR) gives familiar and newly discovered correct outputs the same
12 reward [Guo et al., 2025]. Accuracy can therefore improve while verified outputs narrow (Figure 1).
13 We study how to keep discovered successes available: store verified outputs by identity and revisit
14 them during training. This separates finding a successful behavior from preserving alternatives to it.

2 MODEBENCH makes output identity executable

MODEBENCH assigns each accepted output an exact, executable identity: a canonical key whose distinct values define verified *modes* (Figure 2). Keys capture computation routes in Countdown and MathIR, tested program behaviors in Python, and valid configurations in Graph and PantryPlan. Cosmetic aliases merge; distinct routes can share an answer (key rules: Appendix B). This measures task-defined output support; it does not establish diversity of latent reasoning strategies.

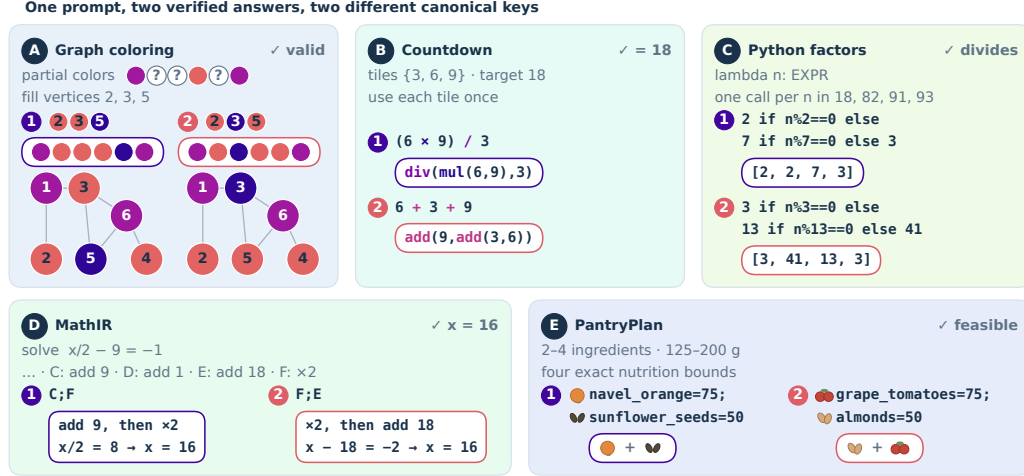


Figure 2: **One prompt, different verified outputs.** Canonical keys identify valid configurations, computation routes and tested behaviors. They merge cosmetic aliases, without identifying latent reasoning.

3 ReplayMaxRL: learn from new samples and remember successes

ReplayMaxRL combines MaxRL’s sampling-aware learning from new samples [Tajwar et al., 2026] with replay of earlier verified successes (Figure 3).

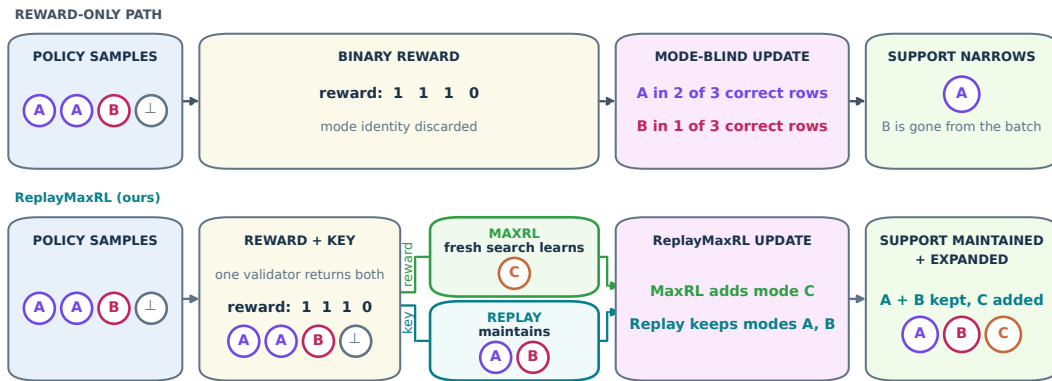


Figure 3: **Discovery and preservation use the same verified samples.** MaxRL learns from fresh rewards; replay revisits discovered modes. Memory cannot protect a mode before it is discovered.

For bank $\mathcal{B}_x$ with one policy-generated exemplar e_c per admitted key, replay averages token-normalized log likelihoods:

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (1)$$

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

	$\Delta \text{ pass@8}$						$\Delta \text{ distinct@8}$					
Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56
	Graph	Countdown	Python	MathIR	Pantry	Average	Graph	Countdown	Python	MathIR	Pantry	Average

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

	$\Delta \text{ pass@8}$						$\Delta \text{ distinct@8}$					
Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04
	Graph	Countdown	Python	MathIR	Pantry	Average	Graph	Countdown	Python	MathIR	Pantry	Average

Figure 4: **Replay improves correctness and verified support across scales.** (A) ReplayDr.GRPO minus Dr.GRPO. (B) Qwen2.5-0.5B alternatives and the initial model minus Dr.GRPO. Daggers indicate four pairs; otherwise five. Averages use shared seeds; intervals: Appendix E. Models: 0.5B, 1B, 3B.

- 26 Equal key weights keep rare discoveries active. ReplayDr.GRPO replaces MaxRL with Dr.GRPO
 27 [Liu et al., 2025], a debiased GRPO [Shao et al., 2024]. Full update: Appendix D; idealized retention
 28 guarantee: Appendix J.4.
- 29 Related work on collapse and replay appears in Appendix I.

30 4 Does memory preserve verified output support?

- 31 We report the chance of any correct output (pass@8), mean distinct verified modes (distinct@8),
 32 and their difference, *extra modes*. Breadth remains coupled to correctness.
- 33 Five registered seeds train for eight passes. Each domain has 128 held-out prompts with four eight-
 34 sample draws; none enter the bank. Controls use the same bank code with zero replay gradient.
 35 Figure 4 also compares GRPO, UCPO [Lochab et al., 2026], RLEP-Dr success replay, and semantic
 36 entropy (protocols: Appendix B).
- 37 **Replay improves held-out correctness and support.** ReplayDr.GRPO improves domain-averaged
 38 correctness and support at each model scale (Figure 4). Uniform key weighting adds .32 extra modes
 39 over frequency weighting, across Qwen2.5-0.5B domains (Appendix E.6).
- 40 **Replay improves both Dr.GRPO and MaxRL.** Adding replay improves average correctness and
 41 verified support under both objectives at Qwen2.5-0.5B and Falcon3-1B (Figure 5). The Qwen2.5-3B
 42 summary uses partial seed sets without an interval (Appendix E.2).
- 43 **Replaying stored solutions also helps on harder problems.** Level 2 preserves validators, keys, and
 44 training/evaluation support histograms. Across four complete five-seed factorials, replay improves
 45 mean correctness and support (Figure 6); PantryPlan remains partial. Benefits vary: Countdown adds
 46 .017 extra modes under ReplayMaxRL, but its correctness interval includes zero (Appendix E.2).
- 47 **Hosted models concentrate despite high accuracy.** Seven deployments show few verified modes
 48 across five domains and three levels (Figure 7). A no-reasoning GPT temperature sweep raises
 49 aggregate pass@8 and breadth through $T = 1.5$ (Appendix H.3). These inference-only results do not
 50 establish training or model-size effects.
- 51 **Conclusion and limits.** Replay improves correctness and sampled breadth, but retains only
 52 discovered modes. Fixed-bank scores still have a declining tail (Appendix F.3). Exact mode
 53 probabilities, causal accuracy mechanisms and benefits under changed constraints remain unverified.

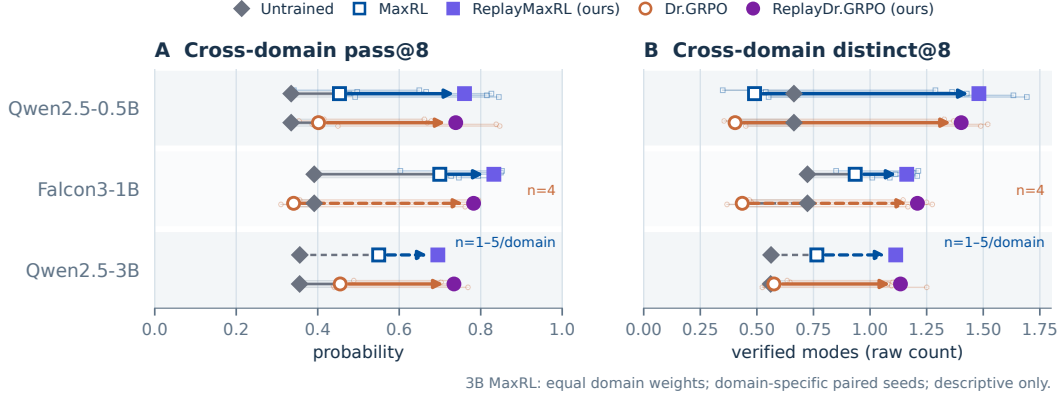


Figure 5: **MaxRL and replay at all three model scales.** Tracks connect initial, trained, and replay models. Complete tracks average within seed; Falcon Dr.GRPO uses four. Qwen2.5-3B MaxRL equally averages five paired domain means ($n = 5, 3, 5, 3, 1$): dashed, descriptive, no interval. Archived models.

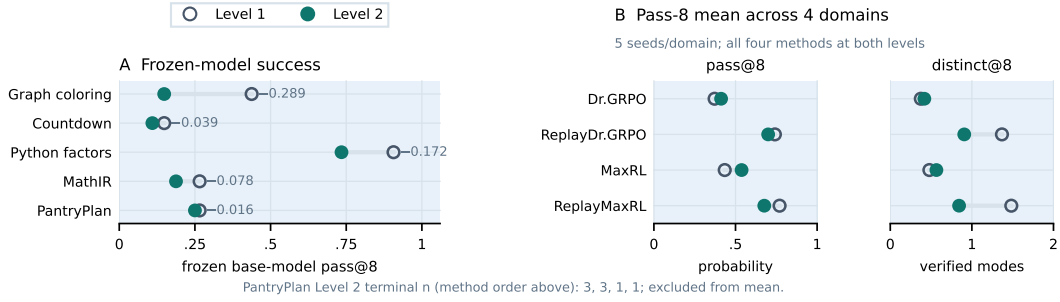


Figure 6: **Replay across difficulty levels with Qwen2.5-0.5B-Instruct.** Open/filled markers: Level 1/2. (A) Frozen-model success. (B) Terminal means weight four complete domains equally within five paired seeds, shared across both levels and all methods. Pantry arm counts remain separate (Appendix E.3). Archived models.

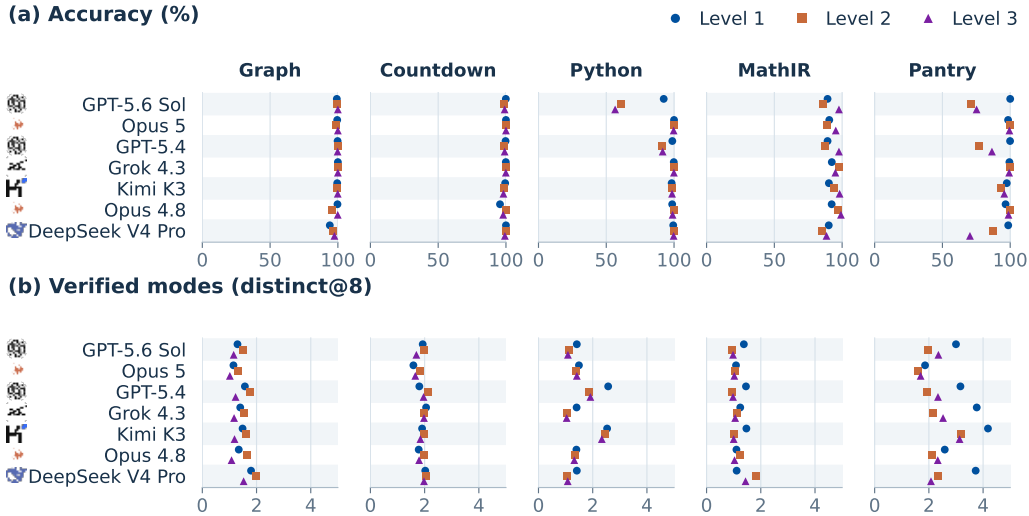


Figure 7: **High accuracy need not imply broad verified outputs.** Each cell uses 128 held-out prompts and eight stateless draws. All responses count after frozen formatting normalization; Opus 5 Python uses separately evaluated revised wording without a system message. Original outputs and protocol comparisons: Appendix H.

References

- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. arXiv preprint arXiv:2505.22617, 2025. URL <https://arxiv.org/abs/2505.22617>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.
- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.

101 Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min
102 Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language*
103 *Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.

104 Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking
105 RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.

106 Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence
107 rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on*
108 *Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 6820–6829.
109 PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.

110 Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL
111 <https://arxiv.org/abs/1504.04909>.

112 Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing
113 neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*,
114 2017. URL <https://arxiv.org/abs/1701.06548>.

115 David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience
116 replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32,
117 2019. URL <https://arxiv.org/abs/1811.11682>.

118 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
119 Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of
120 mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

121 Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode
122 collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL
123 <https://arxiv.org/abs/2601.21669>.

124 Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally
125 can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.

127 Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025.
128 URL <https://arxiv.org/abs/2509.06941>.

129 Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider,
130 Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement
131 learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.

132 Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao
133 Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the
134 base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.

135 Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou.
136 RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.

Supplementary Material

This supplement gives formal metrics, benchmark and protocol details, the replay algorithm, current results and training curves, mechanism checks, core proofs, and reproducibility. Model scales and available paired cohorts match the main figures; detailed histories and theoretical extensions remain in the research archive.

A Formal Metric Definitions

For prompt x with specification s_x , the validator maps an output to a canonical valid key or failure: $V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}$. The learner sees keys only for its generated responses, never the full catalogue $\mathcal{C}_x$ or its size. For $Y \sim \pi_\theta(\cdot | x)$, define

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c | x) = \Pr[V(Y, s_x) = c | V(Y, s_x) \neq \perp].$$

The conditional distribution is defined only when $P_\theta^+(x) > 0$; otherwise all sampling metrics below are zero. Mode collapse means concentration onto fewer valid keys, which can coexist with increasing success probability.

For K independent responses, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

Then $\text{distinct@}K = \mathbb{E}[D_K]$ counts expected unique verified keys in $[0, K]$, whereas $\text{pass@}K = \Pr[D_K > 0]$ and $\text{mean@}K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$. The samplewise inequality $F_K \leq \mathbf{1}[D_K > 0] \leq D_K$ gives $\text{mean@}K \leq \text{pass@}K \leq \text{distinct@}K$, but changes in the count and probability have different units. For a fixed prompt, $\mu_c(x) = \Pr[V(Y, s_x) = c]$ gives

$$\text{pass@}K(x) = 1 - (1 - P_\theta^+(x))^K, \quad \text{distinct@}K(x) = \sum_{c \in \mathcal{C}_x} [1 - (1 - \mu_c(x))^K].$$

These identities apply before averaging over prompts. Raw $\text{distinct@}K$ measures sampled coverage, not total positive-probability support. Uniform independent sampling from N finite actions, with n_c aliases of valid key c , gives $\sum_c [1 - (1 - n_c/N)^K]$ expected keys; open-ended program spaces have no canonical uniform generator.

The terminal co-primary outcomes use $K = 8$: success probability $\text{pass@}8$ and raw $\text{distinct@}8$. Their difference $\mathbb{E}[D_K - \mathbf{1}[D_K > 0]]$ counts extra verified modes beyond the first. This interpretive decomposition remains coupled to correctness and is neither a third primary endpoint nor a success-matched diversity measure.

B Benchmark and Evaluation Protocol

MODEBENCH defines correctness and identity through the same executable validator: invalid outputs receive reward zero and no key; valid outputs receive reward one and a canonical execution key. Neither the complete valid-key catalogue nor its size is supplied to training, replay, scheduling, or stopping. Table 1 specifies the five domains and which surface forms merge. The public dataset preserves each domain’s split roles and construction identity. This paper’s training comparisons use Levels 1 and 2; the separate Level-3 release does not imply completed Level-3 training results.

Table 1: **MODEBENCH**. Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt, exact over the released multi-answer split. Every domain trains on 384 prompts and evaluates on a fixed 128-prompt split; the exact prompt each policy receives is reproduced verbatim in Appendix C for a direct, byte-level audit of the evaluation interface.

Domain	Generated answer / check	Executed key	Aliases that merge	valid modes
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19

B.1 Split identities and executable validation

Every domain uses 384 training and 128 disjoint evaluation prompts. Python materialization independently executes at least two functions with different valid keys for every row; MathIR executes every certified route through the same interpreter used for model outputs.

Table 2: Deterministic materialization identities. Hashes bind the released rows to their executable checks; the full hashes and construction records accompany the data.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96
Gold catalogue supplied to policy	no	no
Training-bank initialization	empty	empty

PantryPlan. The deterministic v2 release has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8 to 45; the 128 evaluation prompts contain 2,328 feasible supports, averaging 18.1875. Training and evaluation row hashes are a4883c5e...a757f and 5d81ba8d...5923. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

B.2 Python execution contract

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean and/or, unary plus/minus/not, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and other identifiers are rejected. Task specifications contain two to eight distinct integers in $[4, 1000]$ and at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm;

the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield $\perp$ and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For every frozen input n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor:  $d_1, \dots, d_m$` ; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

B.3 Matched Level-2 construction

Level 2 changes problem structure while preserving the Level-1 verifier, canonicalizer, training/evaluation sizes, and exact training/evaluation histograms of valid-mode counts. Its 384/128/128 training/development/evaluation identities are disjoint from every Level-1 and Level-2 split. Development histograms also match, except that Pantry scales its 64-row Level-1 histogram by two for 128 Level-2 rows. Thus the evaluated comparison changes difficulty without changing the amount of valid support.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

Before treatment training, frozen Qwen2.5-0.5B evaluation admitted a Level-2 domain only when `pass@8` lay in $[.10, .90]$ and was strictly below its Level-1 value under identical decoding. The released matched-level identity manifest records support histograms, row hashes, construction seeds, disjointness checks, and these admission receipts.

B.4 Training and evaluation settings

Table 3: **Matched run contract.** Settings and systems changes are shared by methods within each model–domain–seed comparison, including the zero-weight replay control.

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .

	Base model	Pinned revision
215	Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
	Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfd
	Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

216 Responses are independent within each evaluation draw. For each training seed, metrics average
217 over all 128 prompts and four draws; raw responses and draw-level records are retained. The co-
218 primary terminal outcomes are `pass@8` and raw `distinct@8`; `mean@8` is diagnostic. Evaluation
219 never supplies training data, bank entries, checkpoint selection, or a stopping rule. Complete paired
220 five-seed blocks receive nominal pointwise 95% Student-*t* intervals; partial primary intersections
221 report exact counts and descriptive means without intervals. Intervals condition on the frozen prompts
222 and evaluation requests. Domain averages are explicitly labeled, and no missing terminal result
223 is replaced by a shallower checkpoint. Source admission and curve gaps follow Appendix K.1;
224 comparison-specific intersections and the replay-weighting bootstrap are specified with their results.

225 C Exact Domain Prompts

226 Each domain uses one system message and one user message, with no few-shot examples. The same
227 template renders training and evaluation for every method. These are the first rows of the frozen
228 evaluation splits, including the exported chat-template wrapper. The checked generator `ops/check_`
229 `paper_domain_prompts.py` reproduces them from the datasets. A two-space continuation indent
230 marks display wrapping: joining it to the previous line with one space recovers the original string.
231 The PantryPlan listing retains its real newlines.

232 C.1 Graph coloring

233 Template `qwen_boxed;` 384 training and 128 evaluation
234 rows in `var/data/graph_coloring_modebench_v2`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3), (3,5),
(3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no edge has the
same color on both endpoints. A partial coloring is 2??1?2, where ? means uncolored.
Keep every shown digit fixed. There are exactly 3 question marks. Return exactly one
final answer and no explanation: exactly 3 digits, each one 1, 2, or 3, for the missing
positions from left to right, inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

235 C.2 Countdown

236 Template `qwen_boxed;` 384 training and 128 evaluation
237 rows in `var/data/exact_countdown_easy3_probe`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use each
given number exactly once. You may use +, -, *, /, and parentheses. Return exactly one
final answer and no explanation: the expression inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

238 C.3 Python factors

239 Template `qwen_boxed;` 384 training and 128 evaluation
240 rows in `var/data/python_factor_modebench_v1`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
```

```

Write one pure Python function with the exact form lambda n: EXPR. An external Python tool
will call it once for each n in [18, 82, 91, 93]. For every call, return an integer d
satisfying 1 < d < n and n % d == 0. Different valid return vectors are different
solution modes. You may use integer literals, n, +, -, *, //, %, comparisons, Boolean
operators, and conditional expressions; calls, imports, attributes, containers, and
other names are forbidden. Return exactly the one-line lambda inside \boxed{} and no
explanation.<|im_end|>
<|im_start|>assistant

```

241 C.4 MathIR action menu

242 Template qwen_boxed; 384 training and 128 evaluation
243 rows in var/data/mathir_action_menu_v1.

```

<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed by
exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c. Action
menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E means
sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations, repeated
states, and paths that do not finish with x isolated are rejected. Return only the IDs
separated by semicolons inside the answer box; for example formatting only: \boxed{A;C}.
Do not return x's numeric value.<|im_end|>
<|im_start|>assistant

```

244 C.5 PantryPlan

245 Template qwen_pantry_support_mask; 384 training and 128 evaluation
246 rows in var/data/pantry_plan_modebench_v2.

```

<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the corresponding
Pantry row in printed order. A trusted environment will choose exact quantities on
precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:
- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal, protein=21.5g,
  fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal, protein=0.74g,
  fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal, protein=0.941g,
  fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant

```

247 D Replay Algorithm

248 For each prompt x , $\mathcal{B}_x$ stores at most M verified policy exemplars, one per canonical key; the
249 serialized cursor τ selects banks for replay. The same update supports either fresh objective.

250 D.1 Fresh task advantages

251 For a group of G fresh responses with verified binary rewards r_i and $m = \sum_i r_i$, Dr.GRPO uses
 252 $A_i = r_i - m/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} Gr_i/m - 1, & m > 0, \\ 0, & m = 0. \end{cases}$$

253 The centered MaxRL estimator [Tajwar et al., 2026] gives failures advantage -1 in mixed groups
 254 and weights each success more strongly when successes are scarce. All-correct and all-failure groups
 255 have zero task advantage. Lemma J.2 gives its harmonic pass@ k mean objective. These advantages
 256 apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

257 D.2 State and optimizer update

Algorithm 1 One ReplayMaxRL optimizer update

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \tau)$
Ensure: Atomically updated $(\theta', \mathcal{B}', \tau')$
 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
 2: **for** $i = 1, \dots, G$ **do**
 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward r_i
 4: Set $c_i \leftarrow \tilde{c}_i$ iff $r_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
 5: **end for**
 6: Compute fresh task advantages A_i^o from $\{r_j\}_{j=1}^G$; for ReplayMaxRL use the formula in Appendix D.1
 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
 8: $(z, \tau') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \tau)$ by a fixed round-robin cursor
 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
 10: For each retained exemplar e_c , compute $s_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} s_c$
 12: **else**
 13: $L_{\text{replay}} \leftarrow 0$
 14: **end if**
 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$
 17: Checkpoint $(\theta', \mathcal{B}', \tau')$ as one atomic state

258 D.3 Admission, capacity, and deterministic scheduling

259 A bank is keyed by the SHA-256 digest of the exact prompt-token sequence. Only loss-active rows
 260 with positive reward and a nonempty validator key qualify. The full group is validated against the
 261 pre-group bank before any new key is committed. For each new key, the lexicographically smallest
 262 normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is
 263 therefore invariant to rollout-row order. The bank keeps one exemplar per key, up to $M = 16$, with
 264 no eviction or replacement. Later keys may enter diagnostics but not replay support when capacity is
 265 full. Prompt-hash collisions or changed tokens under an existing hash are hard errors.

266 At each update, the scheduler sorts nonempty prompt-bank hashes and selects one by a fixed
 267 round-robin cursor; singleton banks are eligible. The cursor is independent of reward, evaluation,
 268 desired support, or entropy, and is serialized with the bank. Stored prompt and response tokens are
 269 materialized unchanged into a right-padded teacher-forcing batch. Prompt and padding positions
 270 are masked. Each exemplar contributes its mean response-token log probability, averaged uniformly
 271 across keys in the selected bank before applying λ_{replay} and the exact $(G - 1)/G^2$ rollout scaling in
 272 Algorithm 1. Neither exhaustive support nor evaluation outputs enter this loss.

273 D.4 Matched controls and recoverable state

274 Controls share validation, bank updates, scheduling, materialization, and the score pass, with the
 275 replay score-gradient vector replaced by exact zeros. Treatment activates only that derivative. Policies

can subsequently differ in samples, bank contents, selected banks, exemplar lengths, and realized computation. Logs retain bank/key counts, replayed token counts, raw and applied losses, score-gradient summaries, cursor phase, and the exact-zero switch. Model and optimizer state, bank counts and exemplar tokens, replay and data cursors, and request-stream state are checkpointed atomically; the loader validates schema, capacity, scheduler and estimator constants, token types, prompt hashes, key/exemplar consistency, and cursor bounds before accepting a resume.

D.5 Fixed Semantic MaxEnt comparator

Fixed Semantic MaxEnt is an entropy-inspired fresh-rollout comparator. For validator-positive canonical outcomes, an empirical predictor combines successful policy history and successful leave-one-out peers. If K keys have total count N , unit pseudocounts and one reserved unseen bucket give

$$\hat{p}(a) = \frac{n(a) + 1}{N + K + 1}, \quad \hat{p}(\text{UNSEEN}) = \frac{1}{N + K + 1}. \quad (2)$$

The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}(a_i), C\} - \mathbb{E}_{\hat{p}}[\min\{-\log \hat{p}(a), C\}]}{C}, \quad \eta = .10, C = 5. \quad (3)$$

Invalid and unsuccessful rows receive zero. The centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome present in a fresh rollout, but supplies no direct reinforcement to an absent remembered exemplar. Its heterogeneous empirical effects therefore remain a comparison with exploration rather than a retention guarantee.

For the true conditional distribution $q_\theta = p_\theta(\cdot | \mathcal{C})$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (4)$$

This motivates the direction of redistribution, not an unbiased-gradient claim for the empirical estimator. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{A \sim p}[\mathbf{1}\{A \in \mathcal{C}\}(s_A - b) \nabla_z \log p_A] = P \mathbb{E}_{c \sim q}[s_c \nabla_z \log q_c] + (\mathbb{E}_q[s] - b) \nabla_z P.$$

A historical baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla H(q)$. For a single correct key, conditional entropy is zero, yet the implemented unseen bucket can give that key negative semantic advantage. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

E Empirical Results and Comparisons

E.1 Comparison registry

The primary comparisons cross fresh objective (Dr.GRPO or MaxRL) with verified replay, first across three model scales on Level 1 and then at Qwen2.5-0.5B on Level 2. Table 4 separates these comparisons from the supporting controls and replay-weighting ablation. Terminal populations use the September 11 frozen census. The registered Qwen2.5-0.5B weighting aggregate retains its original frozen population; its endpoint values are unchanged in that census.

Table 4: **Comparison scope and available terminal evidence.** A complete block has five registered paired seeds. Primary partial intersections retain their exact counts and receive no interval. UCPO and sparse RLEP-Dr are reported only at the two smaller model scales; their denominators cover all five benchmark domains and five registered seeds at each selected scale.

Comparison	Scope	Terminal coverage	Question
Dr.GRPO / ReplayDr.GRPO	Level 1, all three scales	74/75 pairs; 14/15 complete blocks	Verified memory beyond fresh reward
MaxRL / ReplayMaxRL	Level 1, all three scales	67/75 pairs; 12/15 complete blocks	Memory beyond a stronger fresh objective
Four-method factorial	Level 2, Qwen-0.5B	4/5 complete domains; two additional Pantry Dr.GRPO pairs	Transfer to harder matched tasks
GRPO	Level 1, all three scales	75/75 runs	Reward-standardization control
UCPO	Level 1, 0.5B and 1B	50/50 runs	On-policy advantage redistribution
Sparse RLEP-Dr	Level 1, 0.5B and 1B	47/50 runs	Success memory without uniform key balancing
Fixed Semantic MaxEnt	Level 1, registered scale/domain blocks	10/15 complete blocks	Fresh-rollout semantic exploration
Uniform / frequency replay	Level 1, nine registered blocks	43/45 treatment cells; 7/9 complete blocks	Contribution of replay-key balancing

310 E.2 Terminal comparisons across scales

311 Within each model–domain–seed comparison, methods share initialization, data identities, the eight-
312 pass horizon, evaluation requests, verifier, and replay scheduling code. The replay control runs the
313 same bank and scoring path with zero replay derivative. Resulting policies can build different banks.
314 The admissible intersections are defined in Appendix K.1.

Table 5: **Matched results after eight training passes.** Columns compare Dr.GRPO with Re-
playDr.GRPO after exactly eight training passes. Entries are means over the displayed paired-seed
intersection; n @pass gives its size and training pass. P, M, and D denote pass@8, mean@8, and
distinct@8. Every available source-admissible terminal pair is included, no shallower checkpoint
is carried forward, and no missing result is imputed. Falcon Countdown uses four admissible seeds
(Appendix K.1). Pale green marks a ReplayDr.GRPO mean that is strictly higher than its matched
Dr.GRPO mean. Archived models: Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B.

Model	Domain	n @pass	Dr.GRPO			ReplayDr.GRPO		
			P	M	D	P	M	D
Qwen2.5-0.5B	Graph coloring	5@8.0	.323	.323	.325	.969	.404	2.438
	Countdown	5@8.0	.480	.465	.487	.672	.597	1.637
	Python factors	5@8.0	.172	.172	.172	.528	.507	.571
	MathIR	5@8.0	.512	.464	.513	.796	.688	.829
	PantryPlan	5@8.0	.522	.522	.522	.729	.545	1.539
Falcon3-1B	Graph coloring	5@8.0	.656	.300	1.034	.809	.334	1.717
	Countdown	4@8.0	.325	.267	.415	.632	.343	.838
	Python factors	5@8.0	.000	.000	.000	.979	.722	1.107
	MathIR	5@8.0	.095	.068	.097	.736	.292	.756
	PantryPlan	5@8.0	.633	.633	.633	.751	.560	1.576
Qwen2.5-3B	Graph coloring	5@8.0	.682	.360	1.060	.836	.349	1.637
	Countdown	5@8.0	.609	.569	.628	.746	.505	.962
	Python factors	5@8.0	.069	.069	.069	.659	.603	.852
	MathIR	5@8.0	.278	.168	.279	.686	.580	.706
	PantryPlan	5@8.0	.638	.558	.841	.744	.546	1.514

Figure 8 resolves the domain-specific results behind the main cross-domain display. The 3B MaxRL pairs use Graph/Countdown/Python/MathIR/Pantry counts 5/3/5/3/1; the Dr.GRPO pair uses five seeds except Falcon Countdown, which uses four. These populations remain separate: a factorial interaction requires the common four-arm intersection, not differences of aggregates with different seeds.

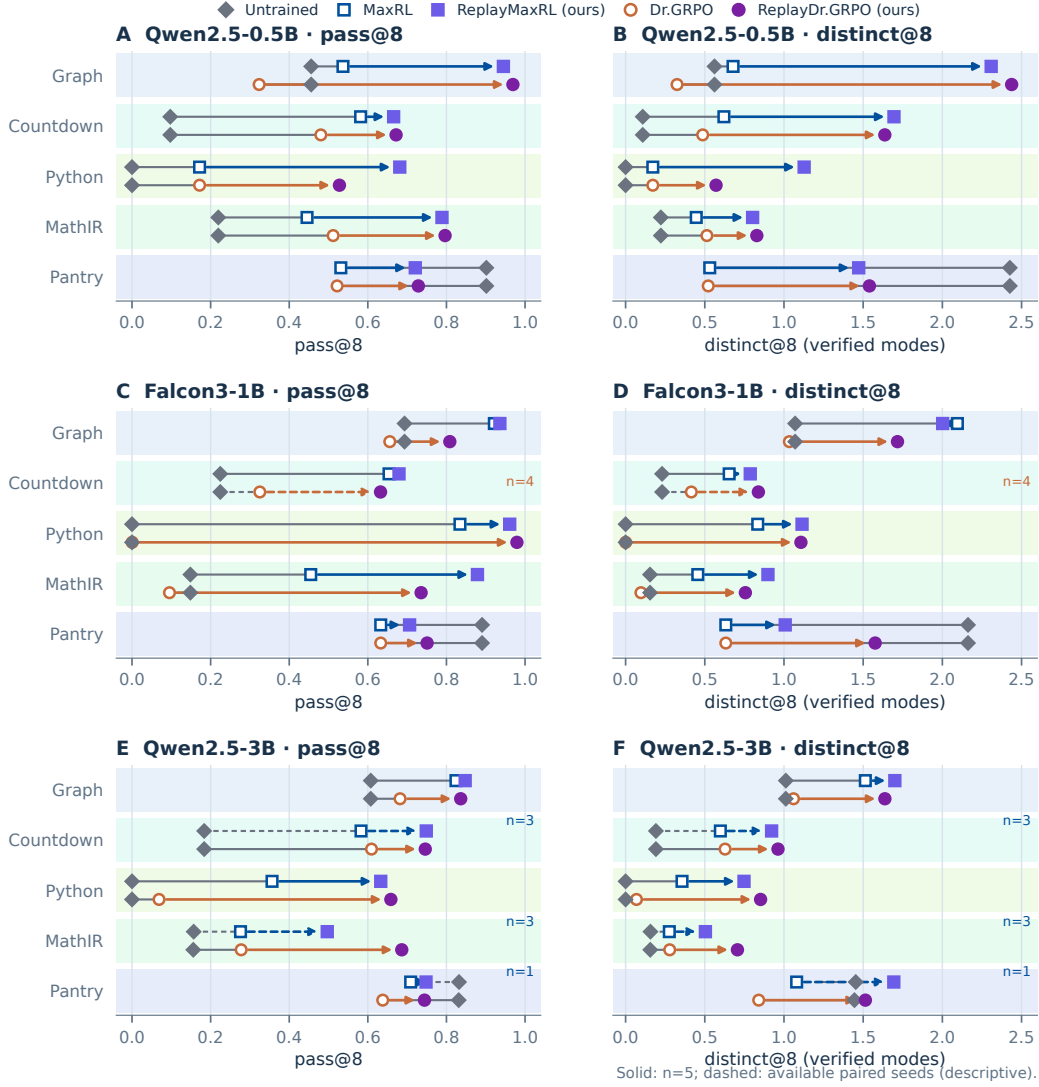


Figure 8: **Per-domain results behind the cross-domain averages in Figure 5.** Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show correctness and raw support. MaxRL–ReplayMaxRL pairs retain five terminal seeds throughout the two smaller models. Qwen2.5-3B uses $n = 5, 3, 5, 3, 1$ in domain order; dashed connectors mark partial blocks. Dr.GRPO–ReplayDr.GRPO uses four admissible seeds for Falcon Countdown and five elsewhere. Domain tints match Figure 2. Replay arrows, trained-arm markers, and untrained references use the same encoding as the main figure. The E118 model index groups the archived exports by model and domain.

Write $P = \text{pass}@8$, $D = \text{distinct}@8$, and $B = D - P$. The derived B counts verified modes beyond the first correct outcome in each sampled set and remains coupled to correctness. The endpoint tables report exact paired terminal effects. A dash denotes no observed pair; partial means retain $n/5$ without an interval. Complete-block intervals are nominal 95% estimates, without multiple-comparison or repeated-look adjustment. All underlying arm means and seed effects remain in the adjacent machine-readable records.

Table 6: **ReplayMaxRL effects across model scales at pass 8.** All effects are ReplayMaxRL minus MaxRL on the displayed paired seeds. Brackets give paired Student- t intervals for ΔB in complete blocks. The machine-readable report also retains both arm means, all per-seed effects, and intervals for P and D .

Model	Domain	Replay arm	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	ReplayMaxRL	5/5	+0.408	+1.630	+1.221 [+0.926, +1.517]
Qwen-0.5B	Countdown	ReplayMaxRL	5/5	+0.084	+1.074	+0.990 [+0.859, +1.121]
Qwen-0.5B	Python	ReplayMaxRL	5/5	+0.509	+0.955	+0.446 [-0.156, +1.048]
Qwen-0.5B	MathIR	ReplayMaxRL	5/5	+0.343	+0.356	+0.013 [+0.008, +0.017]
Qwen-0.5B	PantryPlan	ReplayMaxRL	5/5	+0.189	+0.941	+0.751 [+0.520, +0.982]
Falcon-1B	Graph	ReplayMaxRL	5/5	+0.014	-0.093	-0.107 [-0.220, +0.005]
Falcon-1B	Countdown	ReplayMaxRL	5/5	+0.025	+0.133	+0.108 [+0.082, +0.134]
Falcon-1B	Python	ReplayMaxRL	5/5	+0.127	+0.280	+0.153 [-0.272, +0.578]
Falcon-1B	MathIR	ReplayMaxRL	5/5	+0.424	+0.443	+0.019 [+0.006, +0.031]
Falcon-1B	PantryPlan	ReplayMaxRL	5/5	+0.073	+0.375	+0.302 [+0.034, +0.569]
Qwen-3B	Graph	ReplayMaxRL	5/5	+0.023	+0.186	+0.163 [+0.021, +0.305]
Qwen-3B	Countdown	ReplayMaxRL	3/5	+0.166	+0.324	+0.158
Qwen-3B	Python	ReplayMaxRL	5/5	+0.277	+0.391	+0.115 [+0.080, +0.149]
Qwen-3B	MathIR	ReplayMaxRL	3/5	+0.221	+0.228	+0.007
Qwen-3B	PantryPlan	ReplayMaxRL	1/5	+0.039	+0.613	+0.574

At Qwen2.5-0.5B, ReplayMaxRL adds 1.221, .990, and .751 extra modes on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph instead has a negative mean raw-support effect (-.093, interval [-.229, +.043]), and its correctness effect is inconclusive (+.014, [-.016, +.044]). Positive cross-domain effects therefore do not imply a breadth gain in every domain. The complete Qwen2.5-3B Graph and Python blocks have positive extra-mode intervals while their correctness intervals include zero. The three partial 3B domains retain their own seed sets; the main figure’s equally weighted domain average is descriptive.

E.3 Terminal comparison on the harder level

Figure 6 uses Graph, Countdown, Python, and MathIR: each has all five seeds at step 3,072 for both levels and all four methods. Means average those four domains equally within each seed, then the five seed means. This is a descriptive 20-cell comparison, not a five-domain estimate; the two levels have separate held-out prompts. At Level 2, PantryPlan has three Dr.GRPO, three ReplayDr.GRPO, one MaxRL, and one ReplayMaxRL terminal endpoint. Its only terminal pairs are Dr.GRPO/replay seeds 43 and 46, with no common four-arm seed. Independently available arm means do not estimate paired effects.

Table 7: **Replay effects on the harder Level-2 tasks at pass 8.** MaxRL and Dr.GRPO identify the fresh objective; each effect is its replay variant minus the corresponding objective without replay. Each row uses its own pairwise intersection, which can exceed the four-method intersection. Brackets give paired Student- t intervals for ΔB only when all five registered pairs are available.

Model	Domain	Objective	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Dr.GRPO	5/5	+0.561	+1.019	+0.458 [+0.394, +0.522]
Qwen-0.5B	Graph	MaxRL	5/5	+0.305	+0.668	+0.363 [+0.280, +0.446]
Qwen-0.5B	Countdown	Dr.GRPO	5/5	+0.025	+0.045	+0.020 [-0.014, +0.055]
Qwen-0.5B	Countdown	MaxRL	5/5	+0.040	+0.057	+0.017 [+0.001, +0.032]
Qwen-0.5B	Python	Dr.GRPO	5/5	+0.075	+0.400	+0.325 [-0.228, +0.878]
Qwen-0.5B	Python	MaxRL	5/5	+0.037	+0.212	+0.175 [-0.295, +0.645]
Qwen-0.5B	MathIR	Dr.GRPO	5/5	+0.493	+0.493	+0.000 [+0.000, +0.000]
Qwen-0.5B	MathIR	MaxRL	5/5	+0.173	+0.175	+0.002 [-0.003, +0.006]
Qwen-0.5B	PantryPlan	Dr.GRPO	2/5	+0.234	+0.367	+0.133
Qwen-0.5B	PantryPlan	MaxRL	0/5	—	—	—

Level-2 Graph provides the clearest replay effects on correctness and extra modes under both objectives. Countdown’s ReplayMaxRL extra-mode effect is small (+.017, interval [+0.001, +0.032]), while its correctness and raw-support intervals include zero. Python and MathIR have positive mean correctness and raw-support effects with intervals including zero; MathIR’s extra-mode effects are near zero. Pantry’s two-seed Dr.GRPO replay contrast is +.234 on P , +.367 on D , and +.133 on B , without an interval.

For each common seed and endpoint Y , the replay interaction is

$$I = (Y^{\text{ReplayMaxRL}} - Y^{\text{MaxRL}}) - (Y^{\text{ReplayDr.GRPO}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises P relative to Dr.GRPO by +.233 ([+.140, +.325]), while the replay interaction on P is -.256 ([-.329, -.183]). This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 8: **Fresh-objective effects and replay interactions on Level 2.** All three contrasts within a domain use the same four-arm seed intersection. The interaction is (ReplayMaxRL – MaxRL) – (ReplayDr.GRPO – Dr.GRPO). Each entry gives its mean and, only for a complete five-seed block, a paired unadjusted 95% Student- t interval. Archived four-arm exports: E119 model index.

Domain	n	Contrast	ΔP	ΔD	ΔB
Graph	5	MaxRL – Dr.GRPO	+.233 [+0.140, +0.325]	+.322 [+0.162, +0.482]	+.089 [+0.019, +0.159]
Graph	5	ReplayMaxRL – ReplayDr.GRPO	–.023 [–0.062, +0.016]	–.029 [–0.198, +0.140]	–.006 [–0.142, +0.130]
Graph	5	Replay $\times$ MaxRL interaction	–.256 [–0.329, –0.183]	–.351 [–0.446, –0.255]	–.095 [–0.186, –0.003]
Countdown	5	MaxRL – Dr.GRPO	–.014 [–0.073, +0.045]	–.014 [–0.073, +0.045]	+0.000 [+0.000, +0.000]
Countdown	5	ReplayMaxRL – ReplayDr.GRPO	+0.002 [–0.008, +0.012]	–.002 [–0.047, +0.044]	–.004 [–0.042, +0.035]
Countdown	5	Replay $\times$ MaxRL interaction	+0.016 [–0.047, +0.078]	+0.012 [–0.068, +0.092]	–.004 [–0.042, +0.035]
Python	5	MaxRL – Dr.GRPO	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]	+0.000 [+0.000, +0.000]
Python	5	ReplayMaxRL – ReplayDr.GRPO	–.037 [–0.232, +0.157]	–.188 [–0.988, +0.613]	–.150 [–1.014, +0.713]
Python	5	Replay $\times$ MaxRL interaction	–.037 [–0.232, +0.157]	–.188 [–0.988, +0.613]	–.150 [–1.014, +0.713]
MathIR	5	MaxRL – Dr.GRPO	+0.283 [–0.193, +0.759]	+0.283 [–0.193, +0.759]	+0.000 [+0.000, +0.000]
MathIR	5	ReplayMaxRL – ReplayDr.GRPO	–.036 [–0.158, +0.085]	–.035 [–0.158, +0.088]	+0.002 [–0.003, +0.006]
MathIR	5	Replay $\times$ MaxRL interaction	–.320 [–0.864, +0.225]	–.318 [–0.860, +0.224]	+0.002 [–0.003, +0.006]
PantryPlan	0	MaxRL – Dr.GRPO	—	—	—
PantryPlan	0	ReplayMaxRL – ReplayDr.GRPO	—	—	—
PantryPlan	0	Replay $\times$ MaxRL interaction	—	—	—

E.4 Accuracy and verified modes throughout training

The following curves include every primary method and model scale used in the main paper. Each objective pair uses its domain-specific terminal seed intersection as one fixed cohort throughout

355 training. A checkpoint is plotted only when all members have complete, valid evaluations for both
 356 methods. Missing or conflicting checkpoints leave gaps; seed counts never shrink to extend a mean.
 357 Four fixed-seed draws of eight samples on the same 128 held-out prompts define each evaluation.
 358 Accuracy and mode curves use identical seeds and checkpoints, without smoothing or outcome-based
 359 selection.

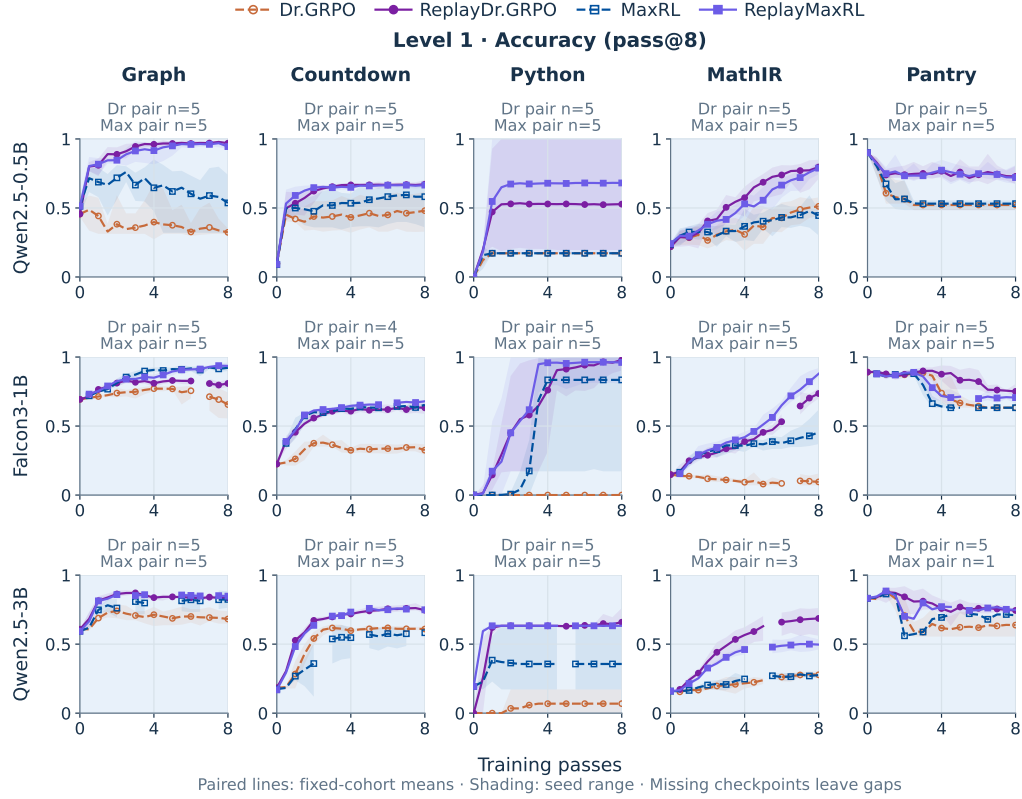


Figure 9: **Accuracy throughout training at all three model scales.** Rows are Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns are the five domains. Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL use the same method colors as Figure 5. Lines are fixed-cohort means of pass@8; bands are seed ranges, not confidence intervals. Counts identify each objective’s terminal paired seeds. Qwen2.5-3B MaxRL retains Graph/Countdown/Python/MathIR/Pantry counts 5/3/5/3/1; Falcon Countdown Dr.GRPO/replay uses four. Gaps indicate missing or conflicting checkpoints.

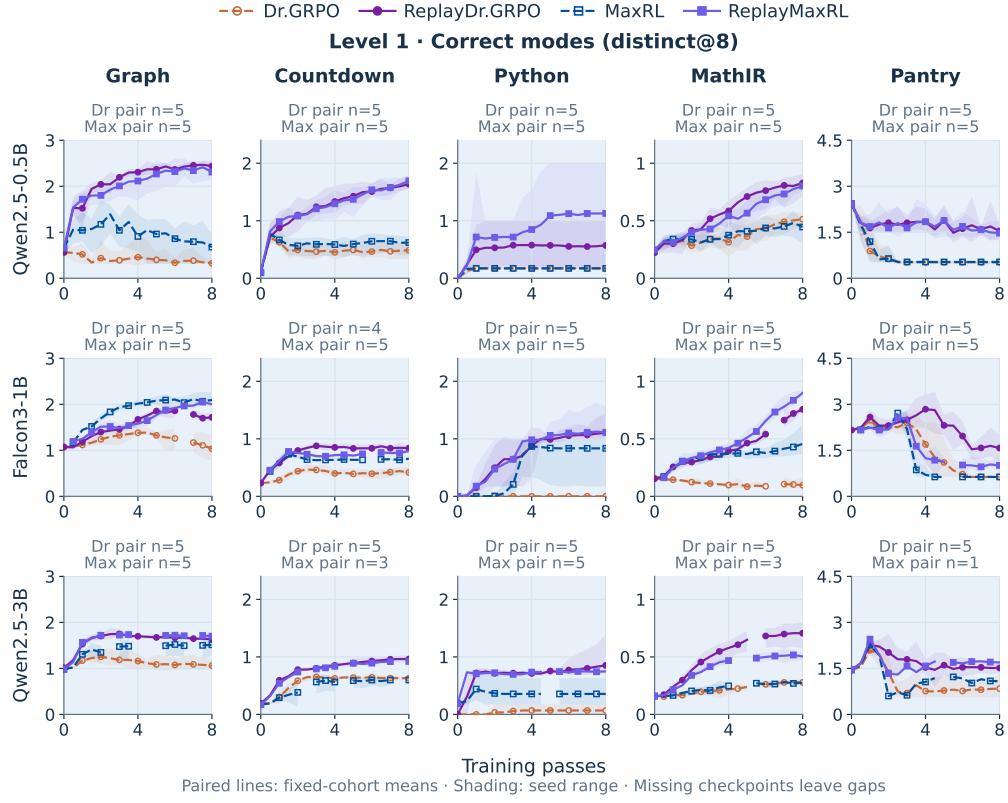


Figure 10: **Verified modes throughout training at all three model scales.** The same model rows, methods, seeds, and checkpoints as Figure 9, now measuring distinct@8. Lines are descriptive fixed-cohort means and bands show seed ranges. A count of one has no band. Domain-specific vertical scales are shared across model rows; absolute heights across different domains should be read against their labeled axes. Missing evaluations are never carried forward or joined across a gap.

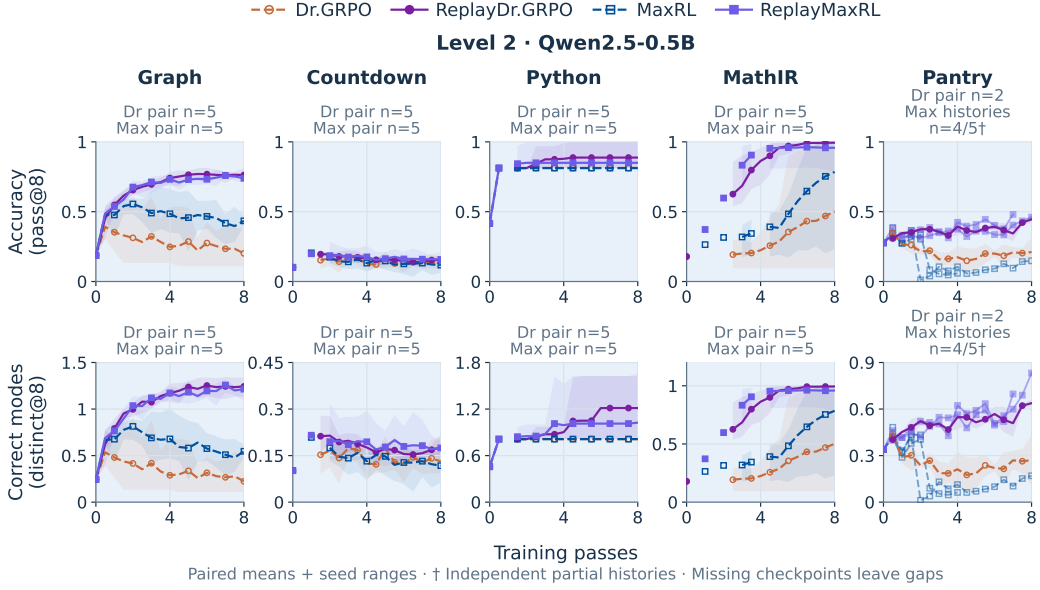


Figure 11: **Accuracy and verified modes during Level-2 training.** Qwen2.5-0.5B is the model tested at this level. Rows show pass@8 and distinct@8 for all four primary methods. Graph, Countdown, Python, and MathIR contain five terminal paired seeds per objective. PantryPlan’s Dr.GRPO/replay means use their two terminal paired seeds; additional MaxRL and ReplayMaxRL histories are individual, unpaired observations, marked separately without a mean or band. They do not estimate a paired replay effect. Both metrics follow the same evaluation rules, with gaps at missing checkpoints and no extrapolation to later training passes.

360 E.5 Supporting comparators

361 GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse
 362 RLEP-Dr replays verified successes without uniform canonical-key weighting. Fixed Semantic
 363 MaxEnt supplies a fresh-rollout exploration incentive (Appendix D.5). These are controls for
 364 alternative explanations of the retention results.

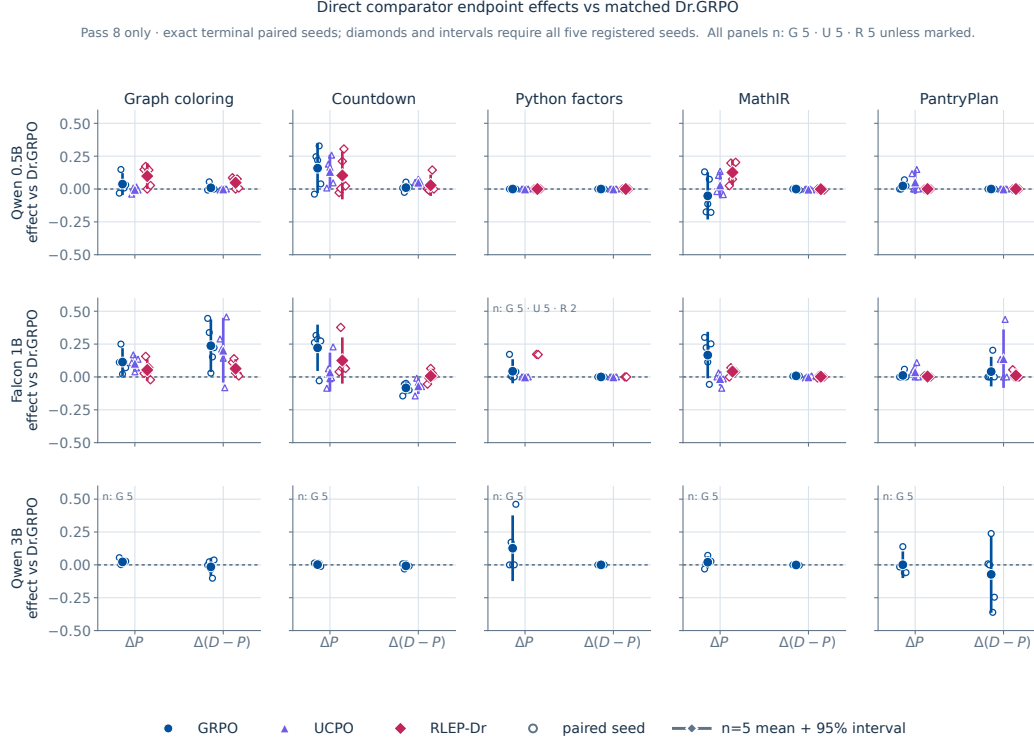


Figure 12: **Direct alternatives produce local rather than uniform final gains.** Each effect subtracts the same-seed Dr.GRPO control from the comparator after eight passes. Effects are shown for pass@8 and correctness-adjusted breadth. Circles are paired seeds; diamonds and 95% Student- t intervals appear only for complete five-seed blocks. Exact n is printed, and unsupported comparisons remain blank rather than being extrapolated or imputed at any seed.

365 The alternatives give local rather than uniform benefits. Qwen-0.5B Countdown UCPO raises
366 P by $+0.133$ ($[+0.005, +0.262]$) and B by $+0.051$ ($[+0.031, +0.071]$), whereas its MathIR effects are
367 inconclusive. MathIR RLEP-Dr raises P by $+0.126$ ($[+0.030, +0.222]$) with neutral extra modes
368 ($[-0.002, [-0.006, +0.003]]$); its Countdown effects and Falcon Pantry effects are inconclusive. The
369 full five-seed 3B plain-GRPO blocks likewise have intervals including zero on both correctness and
370 extra modes. RLEP’s frozen offline pool was collected at temperature $.7$, top- $p = .95$; all reported
371 evaluations use the common temperature-1, top- $p = 1$ contract. An offline pool versus an online
372 bank remains an algorithmic difference.

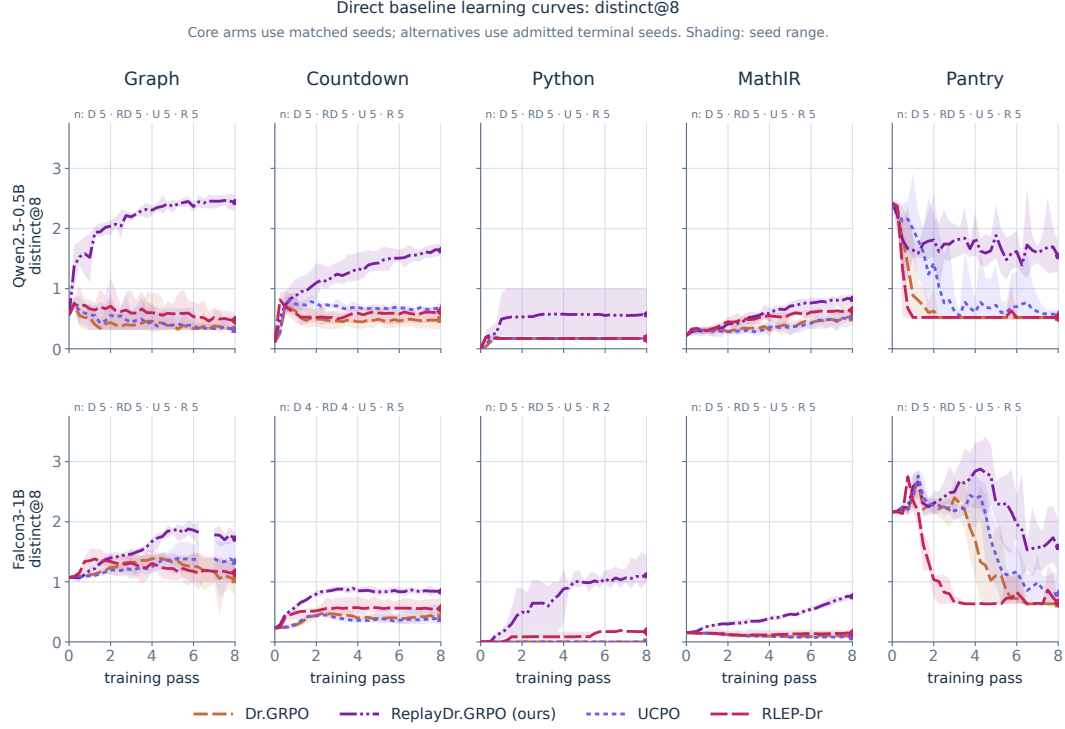


Figure 13: **Verified-mode trajectories for the supporting comparators.** Rows show Qwen2.5-0.5B and Falcon3-1B; columns are the five domains. UCPO and sparse RLEP-Dr are compared with Dr.GRPO and ReplayDr.GRPO at these two smaller scales. Labels give exact method-specific seed counts. Lines are descriptive distinct@8 means and bands are seed ranges. The corresponding accuracy curves use identical populations in Figure 14.

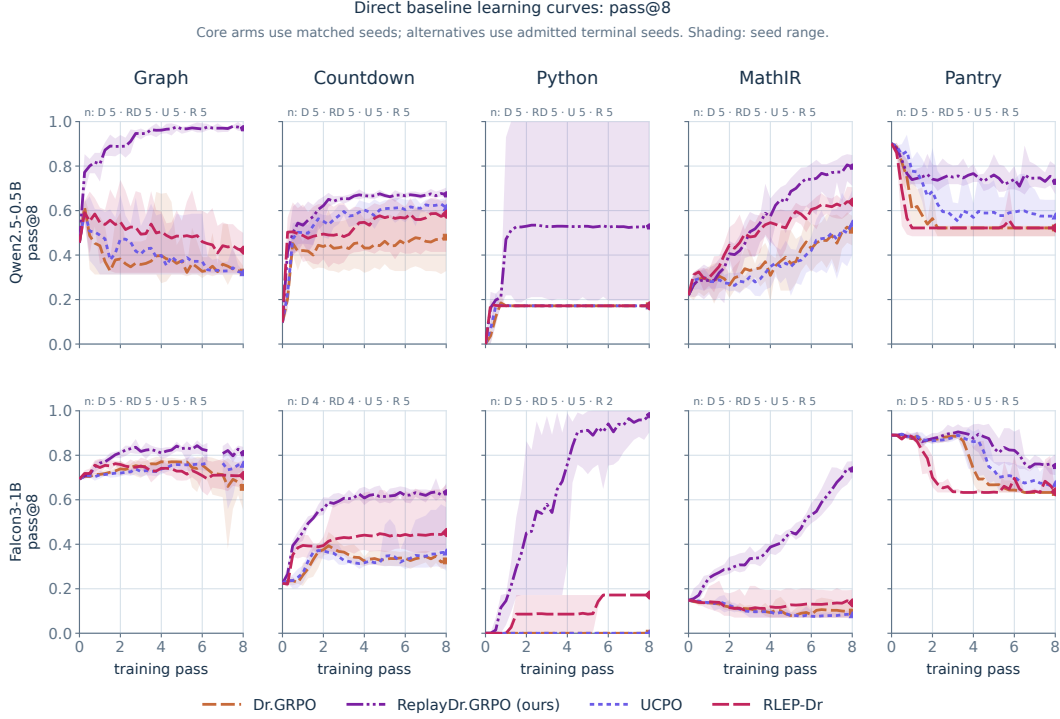


Figure 14: **Accuracy trajectories for the supporting comparators.** The same model rows, domains, methods, terminal seeds, and registered checkpoints as Figure 13, now measuring pass@8. Bands show the seed range and the vertical axes span $[0, 1]$ throughout. MaxRL and ReplayMaxRL at all three scales are compared on both metrics in Figures 9 and 10.

373 E.6 Uniform versus fresh-frequency replay

374 The weighting ablation retains the same bank construction and update code, changing only the within-
 375 bank target: uniform weights over retained keys versus weights proportional to validator-positive
 376 fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below
 377 favor uniform weighting. The registered co-primary contrasts are ΔB and ΔP ; the five-domain
 378 Qwen2.5-0.5B aggregate first averages domain effects within each matched seed, giving equal domain
 379 weight.

380 Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired
 381 training seeds and use linearly interpolated 2.5th and 97.5th percentiles. A resampled seed carries its
 382 complete domain vector when estimating the five-domain aggregate. The endpoints and aggregate
 383 were preregistered; the exact seed-resampling and percentile conventions are analysis choices. These
 384 nominal intervals quantify seed variability within the fixed benchmark, without resampling prompts
 385 or domains or adjusting across comparisons.

Table 9: **Uniform versus fresh-frequency replay at pass 8.** Effects are uniform minus frequency weighting. Brackets give the paired percentile bootstrap interval for ΔB over all 5^5 ordered resamples in complete blocks. Every observed treatment endpoint in this snapshot passes the persisted replay-weight telemetry audit. Partial blocks retain exact seed counts. Paired model exports.

Model	Domain	Weighting	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Uniform	5/5	+0.066	+0.662	+0.596 [+0.513, +0.687]
Qwen-0.5B	Countdown	Uniform	5/5	+0.022	+0.661	+0.639 [+0.544, +0.733]
Qwen-0.5B	Python	Uniform	5/5	-0.077	-0.040	+0.037 [-0.012, +0.123]
Qwen-0.5B	MathIR	Uniform	5/5	-0.005	+0.014	+0.019 [-0.001, +0.038]
Qwen-0.5B	PantryPlan	Uniform	5/5	+0.043	+0.341	+0.298 [+0.174, +0.400]
Falcon-1B	Graph	Uniform	5/5	+0.045	+0.278	+0.233 [+0.182, +0.320]
Falcon-1B	PantryPlan	Uniform	5/5	-0.021	-0.167	-0.146 [-0.564, +0.414]
Qwen-3B	Graph	Uniform	4/5	-0.006	+0.063	+0.070
Qwen-3B	PantryPlan	Uniform	4/5	+0.011	+0.232	+0.222

The registered Qwen2.5-0.5B five-domain effect is +.318 extra modes per prompt ([+.272, +.358]), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The correctness effect is +.010 ([-.119, +.139]), establishing neither a gain nor equivalence. Python’s correctness effects range from -.781 to +.828 across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes (+.233, [+0.182, +0.320]), but its mean sampled correctness decreases by -.0152 ([-.0214, -.0090]); its P interval includes zero. Falcon Pantry is inconclusive: $\Delta B = -.146$ ([-.564, +.414]) and $\Delta P = -.021$ ([-.063, +.026]). Qwen2.5-3B Graph and Pantry have descriptive extra-mode effects of +.070 and +.222 at four pairs each. All observed treatment endpoints pass the persisted replay-weight telemetry audit. The evidence supports a contribution from key balancing in the stated domains, not a uniform accuracy or breadth advantage across every model–domain block.

Evidence boundary. The primary effects are within-domain, within-scale paired comparisons. They do not establish a scaling law, a positive factorial interaction, or preservation of every previously discovered mode. Partial domains are reported as observed and do not support complete-block generality.

F Collapse and Replay Mechanism Checks

F.1 Breadth before and after verifier-only training

The problem precheck evaluates unmodified Dr.GRPO and GRPO at passes 0 and 8 on all three models and five domains: 150 complete runs, each with four fixed $K = 8$ draws. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. Correctness falls in 10 of the 30 domain–method–scale comparisons, while macro raw `distinct@8` ends above its initial value at Qwen2.5-3B. This distinguishes loss of multiple correct outcomes from the broader question of whether any correct outcome is sampled.

Initial evaluations are measured within each method. Qwen2.5-3B MathIR and Pantry E80-R1 seeds 71–74 carry slightly different initial requests from plain GRPO (.154 versus .162, and .832 versus .828 on P). Those cells therefore have separate initial references.

Graph and PantryPlan contain substantial initial breadth at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-breadth percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. ReplayDr.GRPO does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four admissible seeds. These observations concern sampled held-out support, not the survival histories of particular stored modes.

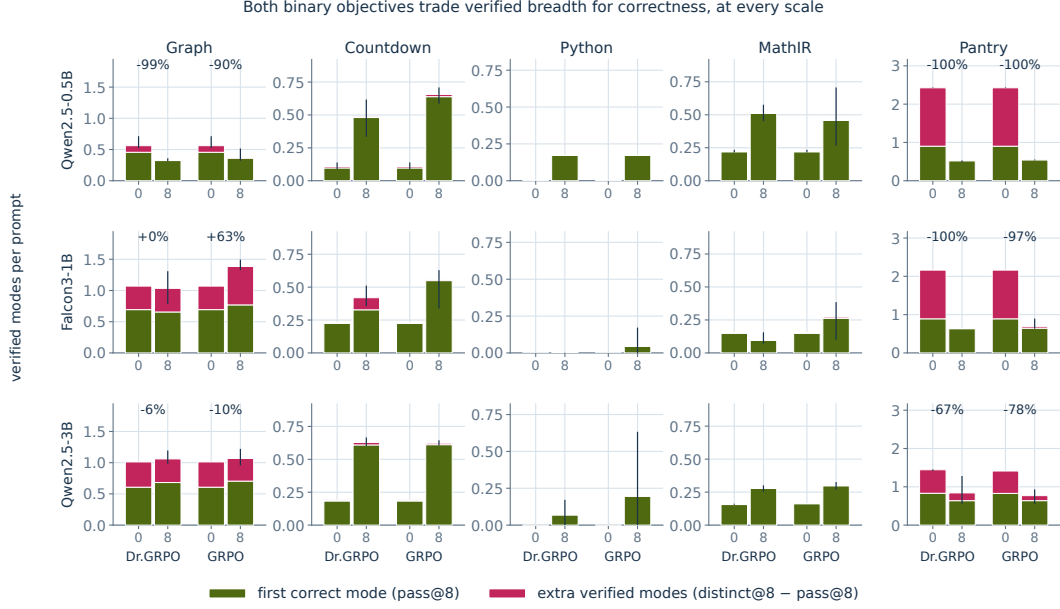


Figure 15: **Both verifier-only objectives reduce verified solution breadth, at every scale.** Bars split distinct@8 into the first correct mode beyond it, at training pass 0 and pass 8, for unmodified Dr.GRPO then GRPO in each panel. All 150 evaluations are complete at five seeds and four fixed $K = 8$ draws; whiskers are the seed range and pass 0 is measured per method. Badges give the change in extra verified modes, printed only where pass-0 breadth exceeds .05 of them; the other three domains start at .011 or less.

Table 10: **Problem precheck, five-domain macro-averages.** Both verifier-only objectives at pass 0 and pass 8, all three scales, five seeds and four fixed $K = 8$ draws per model–domain–method comparison. “Extra modes” is distinct@8–pass@8; “removed” is the fraction of that quantity lost between pass 0 and pass 8. Breadth loss is present at every scale and is far from uniform across scales, and at Falcon3-1B Dr.GRPO macro pass@8 falls as well.

Model	Dr.GRPO			GRPO		
	pass@8	extra modes	removed	pass@8	extra modes	removed
Qwen2.5-0.5B	0.335 → 0.402	0.329 → 0.002	99.4%	0.335 → 0.435	0.329 → 0.006	98.2%
Falcon3-1B	0.391 → 0.342	0.332 → 0.094	71.6%	0.391 → 0.454	0.332 → 0.135	59.4%
Qwen2.5-3B	0.356 → 0.455	0.205 → 0.120	41.5%	0.356 → 0.490	0.199 → 0.101	49.4%

Uniform-action references are meaningful for the enumerable Graph and Pantry representations. They are not defined for open-ended programs or derivations. Pantry’s bit-uniform reference exceeds every final model, so this domain is a support-concentration stress test rather than evidence of complete support coverage.

F.2 Fresh-gradient degeneracy and observed output collapse

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Table 11: **Absolute references for raw distinct@8.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Frozen is the matched pass-0 model; the final two columns are pass-8 means. Dashes avoid pretending that an open-ended program or derivation space has a canonical uniform distribution.

Model	Domain	Uniform	Frozen	Dr.GRPO	ReplayDr.GRPO
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

Domain	Dr.GRPO	ReplayDr.GRPO
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

A raw-response audit of all five Qwen2.5-0.5B Python control seeds establishes observed output collapse directly. At pass 0, a draw has 7.99 unique raw completions per prompt on average. By steps 288–480, every seed produces 32 byte-identical temperature-1 samples per prompt, also identical to the logged greedy completion, on all 128 prompts. This persists through step 3,072. Terminal draws contain 1.00 unique completion per prompt in the control and 1.97 under replay. A collapsed completion can be incorrect: this is one observed completion per prompt, not one correct key everywhere. Aggregate endpoint equality or zero draw spread alone would not establish that conclusion, nor does this finite audit rule out low-probability tails.

These checks explain a vulnerability of the registered sparse binary-reward recipe. They do not establish dominance over methods that keep fresh on-policy signal active. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider-decoding evaluations remain relevant controls beyond the registered factorial.

F.3 Fixed-bank exemplar score trajectories

The fixed-bank study tracks Qwen2.5-0.5B ReplayDr.GRPO on Graph for seeds 43–47. Before step 384’s fresh group, bank membership and fresh counts are frozen; admission then stops while uniform replay continues through 3,072 updates under the same training recipe. Every frozen exemplar is identified by prompt and canonical-key fingerprints, without selection by its score trajectory. Each scheduled replay visit records mean token log probability s and sequence log probability ℓ . Changes compare an identity’s first and final post-freeze visits; observation steps can differ across identities.

All 3,406 frozen identities across 1,575 seed–prompt pairs have finite, aligned scores at every scheduled visit: 8–9 visits per identity and 29,041 observations in total. The pooled median change is $+0.038$ nat/token (95% interval $[+.025, +.048]$), but the 10th percentile is $-.256$ and 2.61% of identities lose more than $.5$ nat/token ($[1.54, 3.77]\%$). The worst intermediate change is -1.328 nat/token. On the sequence scale, the median is $+0.154$ nat/sequence ($[+.104, +.198]$), while 21.99% lose more than $.5$ nat/sequence. Lengths range from 4 to 100 tokens, so the two threshold percentages have different meanings.

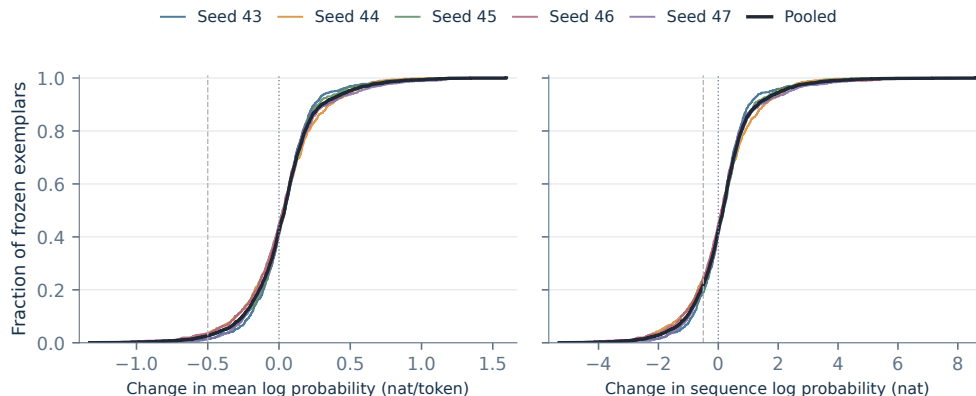


Figure 16: **Fixed-exemplar scores improve centrally but retain a declining tail.** Empirical cumulative distributions show first-to-final post-freeze score changes for every frozen identity, with one curve per seed and the pooled identity distribution. Mean token and sequence scores are separate panels; positive values indicate increased teacher-forced likelihood. Neither panel measures the exact probability of generating a canonical mode during policy sampling.

Pooled summaries weight identities equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap draws (random seed 121), resampling prompts within each original seed and then five seeds with replacement; repeated seeds reuse their within-draw prompt sample. Linear empirical quantiles and percentile intervals are analysis choices. The artifact `paper/results/e121_fixed_bank_survival.json` retains all seed summaries, identity trajectories, source hashes, and audit diagnostics.

These are teacher-forced score surrogates, not exact probabilities of sampling canonical modes. Visits do not cover every intervening update, and the declining tails exclude a claim that every exemplar improves. Without a matched no-replay fixed-bank arm, this study does not identify a causal replay effect. It neither numerically validates Theorem J.6’s probability floor nor certifies a finite-network guarantee. It adds longitudinal measurements of fixed exemplars to the held-out support evidence.

G Inference-Only Concentration in a Hosted Model

This exploratory evaluation measures the current output distribution of GPT-5.6 Sol under the inherited MODEBENCH interface. It is separate from the matched training interventions: it adds no frontier-model training, replay comparison, or controlled model-size contrast. Only the completed, audited deployment enters these tables.

Protocol and provenance. The September 11, 2026 run uses the Azure deployment `gpt-5.6-sol`; saved provider headers identify snapshot `gpt-5.6-sol-2026-07-09`. Each of the five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. The service returns temperature 1.0 and top- p .98; neither setting was overridden. These settings and the hosted natural-language interface differ from the local training evaluation’s token-level grammar constraints.

Level 1 uses the native E118 evaluation splits retained in E124. Level 2 uses the frozen matched r5 construction; Level 3 uses the matched v3 construction admitted by the completed canonical v3 confirmation. Level 3 is an exploratory test extension here, not a third level of the paper’s training intervention. All cells use the multi-answer split. The Level-2/Level-3 support histograms match a separate fixed-reference Level-1 confirmation reserve, which differs from the native Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Exact prompts, dataset identities and hashes, requests, provider-exposed responses, grades, retry receipts, usage and verifier snapshots are retained in the experiment’s evidence directory. All 15,360

intended samples completed, with distinct response identifiers; 107 failed API attempts were retained and recovered by retries. Provider random-number independence and hidden reasoning contents cannot be audited from the exposed responses.

Table 12: **Hosted correctness and concentration under strict grading.** GPT-5.6 Sol, with medium reasoning. Each level weights five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is the percentage of within-prompt correct pairs sharing a mode, pair weighted within each domain. Brackets give pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8 in this paper (called pass@1 in the hosted report). Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise descriptive intervals.

Table 13: **GPT-5.6 Sol under strict executable grading.** Every cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is distinct@8. C is correct-pair collision (%), with 95% prompt-bootstrap interval); U is the uniform-correct-key collision reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Formatting sensitivity. Strict grades remain primary. A secondary typography normalizer was introduced after inspecting the first 15 responses. It fixes a frozen set of presentation conventions, including escaped Python/LaTeX syntax, and reuses the original executable validators without repairing calculations or program logic. It recovers 2,580 additional verified outputs; strict successes

and their keys are retained. This is a post hoc diagnostic, not a replacement benchmark. All 3,072 primary Python responses passed an independent serial recheck with no grade changes. One cold-start timeout in the secondary grading cache was reconciled with repeated serial verification; the original cache and correction audit remain preserved.

Table 14: **Post hoc formatting-normalized diagnostic.** Same samples and metric definitions as Table 13, with fixed presentation normalization before executable verification. The strict table remains primary; normalization was designed after the first 15 responses.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration and level. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized diagnostic gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not a monotonic law within domains. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses particularly clearly: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 correct pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). Graph was selected after the GPT run for this illustration because its correctness is high, support is certified and formatting normalization changes no scores; both tables report all domains.

Interface and claim limits. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Eight draws cannot establish that an unseen mode has zero probability. The evidence supports concentration under the specified prompts and deployment configuration; it identifies neither a training cause nor a causal difficulty or model-size effect.

Reproduction record. The source record is `artifacts/frontier_modebench_gpt56sol_20260911/summary.json`. Its hash, complete cell statistics and run settings are copied into `results/frontier_hosted_20260911.json`; adjacent generated table bodies supply the displayed values. The evidence directory also retains `completion_audit.json`, `prompt_contract_`

541 audit.json, the original requests and responses, and interpretation_notes.md. Hosted-model
 542 results are separate from the training-checkpoint archive and its matched-seed comparisons.

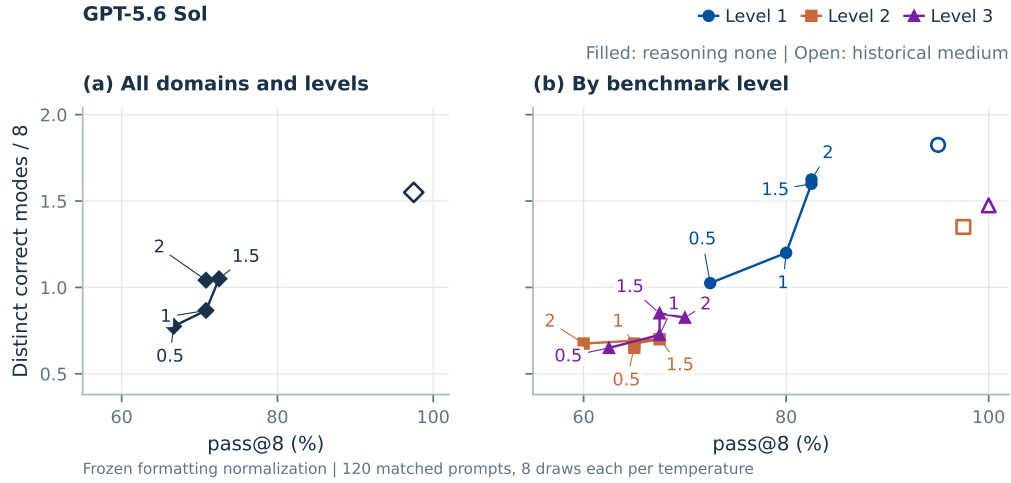


Figure 17: **Temperature traces pass@8 and verified breadth.** GPT-5.6 Sol, with 120 fixed prompts and eight draws each at four temperatures. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical pass@8 is the fraction with at least one correct draw; distinct@8 averages unique correct modes, including zeroes. Connected points use reasoning none and the frozen formatting normalizer. Open markers show the historical medium-reasoning reference on the same prompts (returned $T = 1$), unconnected because its configuration differs. This does not test temperature robustness with medium reasoning. Appendix H.3 gives pointwise uncertainty intervals for every level and all four measured temperatures.

H Comparison across Hosted Deployments

Figure 7 compares verified output breadth across hosted deployments. Its main display applies the frozen formatting normalizer to every deployment and uses the complete revised-wording Opus 5 Python cohort at all three levels. Every response enters its accuracy and mode-count metrics; outputs are not selected by correctness or provider outcome. The other cells use their original prompts. This is a descriptive comparison across the stated task formulations, rather than a common-prompt model ranking.

The tables and Graph figure below preserve every original-protocol result, including failed draws and native refusals. The separate Python comparison reports the original and revised conditions under both grading rules. These records expose the effect of the prompt change without replacing the original evidence. Level 3 remains an exploratory test extension; no hosted-model training or replay intervention is performed. Original deployments enter the comparison only with a passing integrity audit and all 15,360 responses.

Completed hosted comparison. The admitted deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro: 7 complete runs, 107,520 responses, and the same 128 held-out prompts per domain and level with eight stateless samples per prompt. Only runs with a passing completion audit and the identical frozen prompt digest enter these tables. No training, model-side tools or conversation history is added.

Provider configurations. GPT-5.6 Sol: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 5: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. GPT-5.4: hosted API, requested reasoning setting medium, native output limit 8192. Grok 4.3: hosted API, requested reasoning setting medium, native output limit 8192. Kimi K3: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 4.8: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. DeepSeek V4 Pro: hosted API, requested reasoning setting medium, native output limit 8192.

Provider effort labels do not establish equal compute, and native token accounting differs. The saved native request and returned usage determine each configuration; provider defaults remain unspecified whenever native responses omit them.

The appendix retains strict executable grades alongside formatting-normalized scores. The main display uses the same frozen formatting rule for every cell and the complete separately collected revised Opus 5 Python condition; all other displayed cells retain the original request wording. The shared typography diagnostic was developed after the first 15 GPT-5.6 Sol responses and frozen before the prospective comparison runs. It retains its post hoc status for GPT-5.6 Sol. Inherited route preferences and the Level-1 PantryPlan interface change limit cross-level interpretation. Countdown has no certified finite-total-support uniform reference. These data establish neither a training cause nor a causal model-size or difficulty effect. All pair-collision intervals are pointwise; overlapping or nonoverlapping cell intervals are not a paired model-difference test.

Collection interruption: DeepSeek V4 Pro. 13 HTTP-200 bodies had no terminal finish reason or visible answer and were preserved as provider protocol failures. A later storage interruption left 192 logged physical request starts without a saved outcome or usage; their outcome and possible billing remain unknown. The audit registers these attempts separately from the 15,360 terminal draws. Only missing terminal draws were recollected; saved terminal responses, including truncations, were retained. Reported token totals are subtotals of known usage, not zero-cost claims for failed or interrupted attempts. Source-bound recovery sidecars preserve the original receipts and the exact interruption accounting.

H.1 A separate Python prompt sensitivity test

After the original Opus 5 Python refusals, synthetic tasks were used to select a direct arithmetic-expression formulation with no system message. We then collected eight independent draws for every original Python test task: all 384 prompts across three levels, with unchanged adaptive thinking, medium effort, 8,192 output tokens, allowed operators, public inputs and frozen graders. The revised

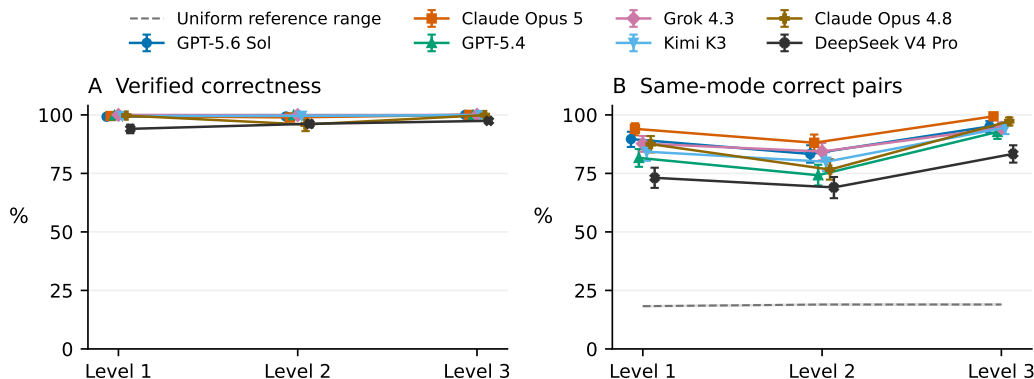


Figure 18: **Verified correctness and concentration in hosted Graph outputs.** Each deployment receives the same 128 prompts per level, with eight stateless samples per prompt. Intervals resample whole prompts. The gray band spans the models’ uniform-correct-key references, conditional on their observed correct draws. Graph was selected after the GPT-5.6 Sol run and before the prospective comparison runs. Level labels do not guarantee increasing difficulty for each model.

594 user instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed
595 input. It does not prescribe a preferred divisor. Both task wording and system-message presence
596 change; collection time also differs. This exploratory follow-up therefore supports a practical prompt
597 repair without isolating its cause. Original refusals and all three residual refusals are retained. The
598 original-protocol appendix tables retain all original Python responses; the main display uses this
599 complete revised condition with an explicit caption note.

Table 15: **Opus 5 Python: original versus plain task wording.** Each row has 1,024 responses. R counts native refusals; A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). Strict and normalized results use the same frozen grader and typography rule. Original Level-2/3 collision is undefined because no prompt has two correct draws.

Condition	L	R	Strict			Normalized		
			A	D	C	A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4
	2	1021	0.3	0.02	–	0.3	0.02	–
	3	1024	0.0	0.00	–	0.0	0.00	–
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9
	2	0	71.8	1.31	84.7	100.0	1.38	85.7
	3	3	76.2	1.38	82.7	99.7	1.41	84.1

600 The new condition has no API failures or token-limit truncations. Strict grading accepts 2,282
601 responses; the frozen rule rescues 787 more by converting escaped percent signs (%) to Python
602 modulo (%). Thus every non-refused answer passes after this formatting-only correction. Normalized
603 collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The residual
604 three refusals occur on two prompts whose other identical requests are accepted. No retry-until-
605 accepted selection is used.

606 The paired analysis resamples whole eight-draw prompt groups across conditions (20,000 replicates).
607 Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2,
608 100.0] percentage points (pointwise 95% intervals). Output draws themselves are independent, and
609 the different correct-pair populations preclude interpreting a raw collision difference as a matched-
610 population prompt effect.

611 The saved sensitivity record binds the original and revised native receipts, request payloads, strict
612 regrade audit and frozen normalization outputs.

Separate first-valid retry diagnostic. After completing the fixed 3,072-response revised-wording cohort, a separate condition retries only its three remaining refused slots, retaining the first verified-valid response. All three succeed on their first additional request, and all reproduce a mode already seen for that prompt. This uses 3,075 requests to select 3,072 valid responses; its 100% accepted-set accuracy follows from selection and is not unconditional model accuracy. The selected set’s `distinct@8` is unchanged at 1.492, 1.375, 1.414 at Levels 1–3. Level-3 correct-pair collision changes from 84.0909% to 84.1518%; the other levels are unchanged. All initial and retry responses remain preserved. The main figure continues to use the complete fixed eight-draw condition, including its three refusals; these selected-validity results are only a separate diagnostic with variable request effort.

H.2 A matched requested-temperature sensitivity test

A separate fixed subset contains eight prompts from each of the five domains at each of three levels: 120 prompts, with eight independent draws per prompt and temperature. Grok 4.3 and Kimi K3 each receive requested temperatures $T = 1.0$ and $T = 1.5$, giving 960 draws per condition and 3,840 in total. The two conditions share task rows and all non-temperature request settings; both temperatures are collected concurrently within each deployment. Every response is retained. This subset is a separate experiment and does not replace any full-cohort result in the main hosted display.

Provider API contracts support the requested controls and the deployments accept the requests, but native responses do not report effective sampling temperature. DeepSeek is excluded from this contrast because its documented thinking-mode interface ignores the temperature setting. These contrasts therefore concern the tested requested settings, not arbitrary temperatures or all deployments.

Tables average all five domains equally within each level; All equally averages all 15 domain–level cells. Confidence intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining each eight-draw group. They are pointwise and have no multiplicity adjustment. Correct-pair collision remains conditional on correct draws and can change its eligible prompt population; an undefined cell makes the corresponding macro undefined.

Observed sensitivity. For Grok, the observed breadth change is +0.050 [-0.058, +0.158] modes, and the collision change is -0.78 [-3.86, +2.12] percentage points. These intervals do not show a clear broadening at the tested setting. The per-response accuracy change is -2.29 [-3.85, -0.94] percentage points. This does not establish invariance to temperature in general.

Kimi returns 696/960 token-limited outputs at $T = 1.5$, compared with 0 at $T = 1.0$. Strict accuracy falls from 96.56% to 21.25%, while raw breadth falls from 1.933 to 0.983 modes. This is a severe token-limit and accuracy failure, not a clean increase in correct-output concentration. The five-domain collision mean is undefined at $T = 1.5$ because some cells have no eligible correct pairs. Neither model has provider-declared refusals in this experiment.

† marks an omitted interval when fewer than all 20,000 bootstrap replicates define the contrast. For Kimi collision, only 11,815 and 11,616 replicates are defined at Levels 1 and 2; the raw conditional percentile summaries remain in the source record. We do not present them as ordinary 95% intervals. Undefined cells are never replaced by zero or silently omitted.

Full domain–level estimates, counts, native outcomes and collision eligibility remain in the source-bound temperature analysis. Countdown has no finite-total-support uniform reference; finite-support sensitivity summaries use the other four domains. The paper record authenticates every included condition summary, native completion audit, frozen normalizer, paired analysis source and sampling-control review, with the complete fixed response cohort retained throughout.

H.3 GPT-5.6 Sol: temperature, pass@8 and verified modes

Figure 17 plots empirical `pass@8` against `distinct@8` for $T \in \{0.5, 1.0, 1.5, 2.0\}$, with reasoning none. Each condition retains the same 120 held-out prompts (eight per domain–level cell), with eight stateless responses per prompt: 960 responses per temperature and 3,840 in total. A prompt contributes one to `pass@8` if at least one of its eight answers is verified correct, and contributes its number of distinct correct canonical keys to `distinct@8`. Zero-success groups and all unsuccessful

Table 16: **Requested-temperature sensitivity: Strict executable grades.** A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.50	-70.00 [-75.62, -63.75]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.12	-13.11 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.06	20.94	-78.12 [-83.75, -72.81]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.33	—	—
	All	A	96.56	21.25	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.25	—	—

Table 17: **Requested-temperature sensitivity: Frozen formatting-normalized grades.** A is per-response accuracy (%), D is distinct@8, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.81	-69.69 [-75.31, -63.44]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.61	-12.62 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.38	20.94	-78.44 [-84.06, -73.12]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.18	—	—
	All	A	96.67	21.35	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.20	—	—

draws remain in the denominator. We compute success directly from the saved groups, not as $1 - (1 - p)^8$ from pooled per-response accuracy; task difficulty varies across prompts.

The deployment rejected non-default temperatures with medium reasoning. The connected curves therefore use the supported reasoning `none` condition. The separate historical medium reference uses the same 120 prompts and eight draws; its requests omitted temperature and receipts returned $T = 1$. It remains unconnected because both reasoning setting and collection time differ. This does not test temperature robustness of the medium condition.

Each level averages five domains equally; All averages all 15 cells. Pointwise 95 percent intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining all eight draws. There is no multiplicity adjustment. A zero-width empirical interval reflects constant observed groups, not certain success on unseen prompts.

Table 18: **GPT temperature frontier: Strict executable grades.** P is solved prompts (% , pass@8); D is distinct@8; A is per-response accuracy (%), retained as a separate diagnostic. Brackets give pointwise 95% intervals. M is the historical medium reference; numeric temperatures all use reasoning `none` and the same fixed eight-draw sampling budget.

T	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.5	1	62.5 [50.0, 75.0]	0.925 [0.675, 1.200]	52.5 [41.2, 63.4]
	2	45.0 [37.5, 55.0]	0.450 [0.375, 0.550]	29.7 [22.5, 36.2]
	3	47.5 [35.0, 60.0]	0.475 [0.350, 0.600]	40.0 [27.8, 52.2]
	All	51.7 [45.0, 58.3]	0.617 [0.517, 0.717]	40.7 [34.8, 46.7]
1.0	1	77.5 [65.0, 90.0]	1.175 [0.950, 1.400]	53.1 [42.2, 63.7]
	2	45.0 [35.0, 55.0]	0.475 [0.350, 0.600]	29.4 [22.2, 36.6]
	3	52.5 [40.0, 65.0]	0.550 [0.400, 0.700]	38.8 [27.8, 50.0]
	All	58.3 [51.7, 65.0]	0.733 [0.642, 0.833]	40.4 [34.7, 46.0]
1.5	1	82.5 [70.0, 92.5]	1.525 [1.250, 1.825]	54.4 [44.1, 64.7]
	2	47.5 [37.5, 57.5]	0.475 [0.375, 0.575]	27.8 [22.2, 33.1]
	3	55.0 [42.5, 67.5]	0.675 [0.475, 0.875]	42.2 [31.2, 53.4]
	All	61.7 [55.0, 68.3]	0.892 [0.775, 1.008]	41.5 [36.0, 46.8]
2.0	1	82.5 [70.0, 92.5]	1.600 [1.325, 1.900]	58.4 [48.1, 68.4]
	2	40.0 [32.5, 47.5]	0.425 [0.350, 0.500]	26.6 [21.2, 31.9]
	3	55.0 [42.5, 67.5]	0.625 [0.475, 0.800]	34.7 [25.6, 44.4]
	All	59.2 [53.3, 65.0]	0.883 [0.767, 1.000]	39.9 [34.9, 44.8]
M	1	95.0 [87.5, 100.0]	1.675 [1.425, 1.925]	79.4 [71.6, 86.6]
	2	75.0 [67.5, 80.0]	1.025 [0.825, 1.225]	59.4 [50.3, 67.2]
	3	85.0 [80.0, 92.5]	1.225 [1.000, 1.475]	71.2 [63.1, 79.7]
	All	85.0 [81.7, 88.3]	1.308 [1.183, 1.442]	70.0 [65.2, 74.6]

Observed temperature frontier. From $T = 0.5$ to $T = 1.5$, normalized pass@8 rises from 66.67% to 72.50%, while distinct@8 rises from 0.775 to 1.050. At $T = 2.0$, the corresponding values are 70.83% and 1.042. Thus $T = 1.5$ is the best observed aggregate point on both axes among these four temperatures. This finite exploratory sweep does not establish a population optimum or a continuous Pareto boundary, and level-specific curves differ. In particular, Level 1 continues gaining modes at the highest temperature even though the overall point declines slightly.

Per-response accuracy nevertheless decreases from 55.52% to 50.21% between the endpoints. That diagnostic is distinct from pass@8: success within eight draws can improve while an individual draw becomes less reliable on these same prompts.

The paired $T = 2.0$ minus $T = 0.5$ contrast is 4.2 [-0.8, 9.2] pass@8 percentage points and 0.267 [0.183, 0.350] distinct correct modes, with pointwise 95% intervals. All prompt-level records, domain estimates, original per-response and collision analyses, and paired comparisons remain in the retained reports for both strict and normalized grading.

Retained outcomes. At $T = 0.5$, 0 of 960 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.0$, 0 of 960 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.5$, 0 of 960 outputs are token-limited and 0 are provider-declared refusals. At $T = 2.0$, 0 of 960 outputs are token-limited and 0 are provider-declared refusals. These outcome counts retain every

Table 19: **GPT temperature frontier: Frozen formatting-normalized grades.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%), retained as a separate diagnostic. Brackets give pointwise 95% intervals. M is the historical medium reference; numeric temperatures all use reasoning none and the same fixed eight-draw sampling budget.

<i>T</i>	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.5	1	72.5 [60.0, 85.0]	1.025 [0.775, 1.300]	67.8 [55.9, 79.1]
	2	65.0 [57.5, 75.0]	0.650 [0.575, 0.750]	46.2 [38.1, 53.8]
	3	62.5 [50.0, 75.0]	0.650 [0.525, 0.775]	52.5 [40.3, 64.4]
	All	66.7 [60.0, 73.3]	0.775 [0.683, 0.875]	55.5 [49.3, 61.7]
1.0	1	80.0 [67.5, 90.0]	1.200 [0.975, 1.425]	67.2 [55.3, 78.4]
	2	65.0 [55.0, 75.0]	0.675 [0.550, 0.800]	44.4 [36.6, 52.2]
	3	67.5 [55.0, 80.0]	0.725 [0.600, 0.875]	50.0 [39.4, 60.9]
	All	70.8 [64.2, 77.5]	0.867 [0.775, 0.967]	53.9 [47.9, 59.6]
1.5	1	82.5 [70.0, 92.5]	1.600 [1.325, 1.900]	67.2 [55.9, 78.1]
	2	67.5 [57.5, 77.5]	0.700 [0.600, 0.800]	39.1 [32.5, 45.6]
	3	67.5 [55.0, 80.0]	0.850 [0.650, 1.050]	50.9 [40.0, 61.9]
	All	72.5 [65.8, 79.2]	1.050 [0.925, 1.175]	52.4 [46.8, 57.9]
2.0	1	82.5 [70.0, 92.5]	1.625 [1.325, 1.925]	68.4 [57.5, 78.8]
	2	60.0 [52.5, 67.5]	0.675 [0.575, 0.775]	37.5 [31.6, 43.1]
	3	70.0 [57.5, 82.5]	0.825 [0.650, 1.000]	44.7 [34.7, 54.7]
	All	70.8 [65.0, 76.7]	1.042 [0.925, 1.158]	50.2 [44.9, 55.3]
M	1	95.0 [87.5, 100.0]	1.825 [1.575, 2.075]	93.4 [86.6, 98.8]
	2	97.5 [92.5, 100.0]	1.350 [1.175, 1.550]	80.9 [73.8, 87.5]
	3	100.0 [100.0, 100.0]	1.475 [1.250, 1.700]	85.0 [80.0, 90.0]
	All	97.5 [95.0, 100.0]	1.550 [1.425, 1.683]	86.5 [82.8, 89.9]

691 registered draw and are separate from the verifier-based correctness and canonical-key measurements
692 above.

693 H.4 Native provider outcomes and undefined concentration

694 Provider-declared refusals and filtering are deployment outcomes, not evidence that the underlying
695 model cannot solve the mathematical task. Empty refusals remain failed draws under the frozen
696 verifier; no requests or grades are overwritten. Classification uses native metadata, not a semantic
697 detector of refusals written as ordinary answer text.

698 Correct-pair collision is undefined when no prompt supplies two correct draws. The five-domain
699 mean is therefore undefined if any domain has no eligible pairs; such entries remain – rather than
700 zero or a mean over fewer domains.

701 GPT-5.6 Sol native completion states: completed: 15,360. Empty visible answers are reported
702 separately from provider refusals.

703 Claude Opus 5 native completion states: end_turn: 12,838, refusal: 2,522. Empty visible answers are
704 reported separately from provider refusals.

705 Claude Opus 5 returns provider-declared refusals for 409, 1021, 1024 of the 1,024 Python requests at
706 Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are cyber: 2454.
707 These are benign factor-selection tasks; the reported deployment policy must be distinguished from
708 concentration among correct Python outputs.

709 GPT-5.4 native completion states: completed: 15,355, incomplete: 5. Empty visible answers are
710 reported separately from provider refusals.

711 Grok 4.3 native completion states: stop: 15,360. Empty visible answers are reported separately from
712 provider refusals.

713 Kimi K3 native completion states: length: 15, stop: 15,345. Empty visible answers are reported
714 separately from provider refusals.

715 Claude Opus 4.8 native completion states: end_turn: 15,355, max_tokens: 5. Empty visible answers
716 are reported separately from provider refusals.

Table 20: **Native service outcomes: GPT-5.6 Sol.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 21: **Native service outcomes: Claude Opus 5.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	409	0	409
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	1021	0	1021
2	MathIR	35	0	35
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	1024	0	1024
3	MathIR	33	0	32
3	PantryPlan	0	0	0

717 Claude Opus 4.8 returns provider-declared refusals for 0, 0, 0 of the 1,024 Python requests at Levels
718 1, 2 and 3, respectively. The native refusal categories on these Python draws are not supplied.
719 These are benign factor-selection tasks; the reported deployment policy must be distinguished from
720 concentration among correct Python outputs.

721 DeepSeek V4 Pro native completion states: length: 632, stop: 14,728. Empty visible answers are
722 reported separately from provider refusals.

723 The result record `results/frontier_comparison_20260911.json` binds every source summary,
724 completion audit, frozen prompt digest, raw-sample digest and primary graded sample digest. Pending
725 deployments are omitted from both the figure and numerical comparisons. The figure was selected
726 after the reference GPT run for its high strict correctness, certified support and lack of formatting
727 corrections, and before collecting the prospective comparison runs. All five domains are retained in
728 the tables, including disagreement across models.

Table 22: **Native service outcomes: GPT-5.4.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	1
2	Graph	0	0	0
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	1
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	2

Table 23: **Native service outcomes: Grok 4.3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 24: **Native service outcomes: Kimi K3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 25: **Native service outcomes: Claude Opus 4.8.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	3
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	1

Table 26: **Native service outcomes: DeepSeek V4 Pro.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	59
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	4
1	PantryPlan	0	0	13
2	Graph	0	0	38
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	59
2	PantryPlan	0	0	80
3	Graph	0	0	24
3	Countdown	0	0	7
3	Python	0	0	0
3	MathIR	0	0	100
3	PantryPlan	0	0	246

Table 27: **Strict level summaries for completed hosted deployments.** Each level weights the five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is correct-pair collision (%; with pointwise 95% whole-prompt intervals), pair weighted within each domain. Provider settings differ; these are descriptive outputs.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	—
Claude Opus 5	3	77.8	79.8	1.07	—
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Table 28: **GPT-5.6 Sol: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Table 29: **Claude Opus 5: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5

Table 30: **GPT-5.4: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–
1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–
2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–
3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 31: **Grok 4.3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 32: **Kimi K3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–
1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–
2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–
3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 33: **Claude Opus 4.8: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5

Table 34: **DeepSeek V4 Pro: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 35: **GPT-5.6 Sol: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Table 36: **Claude Opus 5: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5

Table 37: **GPT-5.4: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 38: **Grok 4.3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 39: **Kimi K3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 40: **Claude Opus 4.8: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5

Table 41: **DeepSeek V4 Pro: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

I Related Work

Repeated sampling and verifier-guided selection depend on the proposal distribution [Brown et al., 2024, Snell et al., 2024]. Prior work studies diversity loss in alignment and reasoning [Kirk et al., 2024, Dang et al., 2025], entropy dynamics [Cui et al., 2025], and high-budget coverage [Yue et al., 2025]. Semantic clustering [Farquhar et al., 2024] addresses open-ended outputs; MODEBENCH instead uses executable identity keys. Sampling-aware objectives promote useful proposal sets [Song et al., 2025, Chen et al., 2025, Li et al., 2025, 2026a,b, Lochab et al., 2026]. MaxRL optimizes best-of- k success [Tajwar et al., 2026], which equals pass@ k for binary rewards. Replay has a long history in reinforcement and continual learning [Lin, 1992, Rolnick et al., 2019]; RLEP reuses successful reasoning traces [Zhang et al., 2025]. Our bank retains one policy exemplar per verified key and weights keys uniformly, connecting persistent success memory to novelty and quality-diversity search [Lehman and Stanley, 2011, Mouret and Clune, 2015].

J Mode Collapse and Verified Replay

This appendix establishes the categorical claims behind the method: centered binary-reward estimators amplify a pre-existing imbalance among correct modes, whereas persistent verified replay supplies a loss barrier for banked modes. The complete-response argument then states what would be needed to transfer that protection to a language model. These results concern idealized update rules; they do not certify the implemented AdamW/PPO trajectories.

J.1 Categorical reduction and mean updates

Fix one prompt and a finite response alphabet $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$, with $m \geq 2$ correct execution modes and $n \geq 1$ incorrect categories. Let $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$. Thus P measures correctness and q the distribution among correct modes. Throughout the mean-flow analysis, logits are finite initially, $0 < P(0) < 1$, explicit reference KL is zero, and the prompt is isolated.

Draw a group $A_1, \dots, A_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[A_i \in \mathcal{C}]$, and let $M = \sum_i R_i$. Both GRPO variants can be written, at the on-policy point where the PPO ratio equals one, as

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(M) \left(R_i - \frac{M}{G} \right) \nabla_z \log p_{A_i}. \quad (5)$$

For Dr.GRPO, $w(M) = 1$ after absorbing the shared positive length scale. For binary-reward GRPO, $w(M)$ is the reciprocal group reward standard deviation on mixed groups. We absorb the same fixed response-length normalizer in both variants; response-dependent row normalization is outside this reduction. In either case $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. We define the entire update as zero on all-equal groups, without evaluating an undefined reciprocal standard deviation; likewise, $w(M)M(G-M)$ below is defined as zero when $M = 0$ or $M = G$. We take explicit reference KL to be zero, as in the experiments.

Lemma J.1 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected group update is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(M)M(G-M)]}{G^2 P(1-P)} > 0. \quad (6)$$

For Dr.GRPO, $c_G(P) = (G-1)/G$. Hence reward centering and binary reward standardization change the speed, but not the expected direction, of the binary-reward policy gradient.

Proof. Condition on the complete binary reward vector, whose sum is M . The score identities give

$$\mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{A_i} \mid R_i = 0] = \nabla_z \log(1-P).$$

There are M correct rows with centered reward $(G-M)/G$ and $G-M$ incorrect rows with centered reward $-M/G$. Therefore the conditional expectation of the unaveraged sum in (5) is

$$w(M) \frac{M(G-M)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(M) \frac{M(G-M)}{GP(1-P)} \nabla_z P.$$

769 The outer factor $1/G$ and expectation over $M \sim \text{Binomial}(G, P)$ yield (6). For $w \equiv 1$, $\mathbb{E}[M(G -$
770 $M)] = G(G - 1)P(1 - P)$, proving the Dr.GRPO value. $\square$

771 The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1)P^j(1-P)^{G-2-j}.$$

772 The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. This establishes the smooth, positive extension to $P = 0, 1$ used in the
773 subsequent flow and convergence arguments.
774

775 The next lemma specializes the practical-estimator analysis of Tajwar et al. [2026, Section 4.3 and
776 Appendix D, Theorem 5] to our categorical notation; we include the derivation for completeness.

777 **Lemma J.2** (Binary MaxRL has the same correctness-gradient direction). *Let MaxRL assign*
778 $A_i^{\text{MaxRL}} = GR_i/M - 1$ *when* $M > 0$ *and zero when* $M = 0$, *as in Section 3. At the on-policy point,*
779 *its expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1-P)^{G-1}}{P} > 0, \quad (7)$$

780 *with continuous endpoint values* $c_G^{\text{MaxRL}}(0) = G-1$ *and* $c_G^{\text{MaxRL}}(1) = 1$.

781 *Proof.* Condition on $M = m > 0$. Each successful row has advantage $(G-m)/m$ and each failed
782 row has advantage -1 . Using the same conditional score identities as above, the expected unaveraged
783 sum is

$$(G-m)(\nabla_z \log P - \nabla_z \log(1-P)) = \frac{G-m}{P(1-P)} \nabla_z P.$$

784 After the outer factor $1/G$ and averaging over $M \sim \text{Binomial}(G, P)$,

$$\mathbb{E}[(G-M)\mathbf{1}\{M > 0\}] = G(1-P) - G\Pr(M=0) = G[(1-P) - (1-P)^G].$$

785 Substitution gives Equation (7). Its endpoint values follow by continuity (or the finite geometric-series
786 identity), which also gives the stated continuous extension at both boundaries. $\square$

787 The same finite-series identity gives the potential used below:

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1-P)^k}{k}. \quad (8)$$

788 In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, $\text{best}@k$ equals $\text{pass}@k$, so this is
789 a harmonic mixture of sampling objectives for $k = 1, \dots, G-1$. The upper limit is $G-1$, not
790 G : the implemented centered estimator drops the entire update on all-failure groups. Subtracting
791 an unconditional score baseline even on those groups instead gives the order- G estimator. This
792 distinction concerns the exact finite-group expectation at the on-policy point; it does not assert
793 unbiasedness of subsequent clipped or adaptive-optimizer updates.

794 J.2 Winner-take-all collapse in the categorical mean flow

795 Consider $\dot{z} = c_G(P) \nabla_z P$, with any of the three smooth positive functions established above. This
796 infinitesimal expected-update model removes sampling noise and retains independently trained
797 Euclidean logits. The conditional dynamics reduce to the standard frequency-dependent replicator
798 system with fitness $f_c(q) = q_c$ [Hofbauer and Sigmund, 1998]. The result is a specialization to
799 the group estimators, not a new general theorem about replicator dynamics. Related correctness-
800 indifferent collapse analyses include Sinha et al. [2026], Lochab et al. [2026].

801 **Theorem J.3** (Standard replicator specialization for binary-reward RL). *Start from finite logits with*
802 $0 < P(0) < 1$. *If the conditional correct-mode distribution* $q(0)$ *has a unique largest coordinate* c^* ,
803 *then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

804 *Thus these GRPO, Dr.GRPO, and binary MaxRL mean flows converge to a single correct execution*
805 *mode for generic initial conditions, even though every mode in* $\mathcal{C}$ *has exactly the same reward.*

806 *Proof.* For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P)p_c(1 - P), \quad \dot{z}_a = -c_G(P)p_aP.$$

807 Also $\dot{P} = c_G(P)\|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G ,
808 together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

809 would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

810 Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

811 If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute
812 velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$.
813 Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

814 On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (9)$$

815 The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so
816 $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

817 Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ;
818 normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

819 If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio
820 argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly
821 uniform correct distribution is therefore a special initial condition. The theorem concerns limiting
822 concentration: finite softmax logits already give positive probabilities at every finite time, so finite-
823 time positivity alone is not a retention result.

824 **Dependence on optimization geometry.** For a smooth parameterization $z = z(\theta)$, Euclidean
825 parameter flow instead induces $\dot{z} = c_G(P)J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses
826 $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization
827 metric. Parameter count alone does not specify this geometry. The empirical scale comparisons also
828 change pretrained models and learning-rate schedules; they neither measure the geometry nor identify
829 the cause of cross-scale differences.

830 J.3 A replay signal when fresh groups supply none

831 **Lemma J.4** (Fresh-group starvation and replay signal). *Fix a categorical policy with correctness*
832 *probability P , and draw $G \geq 2$ fresh responses independently. For GRPO, Dr.GRPO, and the*
833 *implemented centered MaxRL estimator, the probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

834 *Every all-equal group has zero fresh task gradient. Thus h_G is the probability of an opportunity for a*
835 *nonzero task gradient, rather than a lower bound on the gradient magnitude.*

836 *For a fixed bank with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ and $\rho_w > 0$. Extend*
837 *w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is*

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p).$$

838 *It vanishes only at $p = \bar{w}$. In particular, if $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$,*
839 *even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes, as $p \rightarrow \mathbf{e}_b$ at*
840 *one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$. The replay norm*
841 *therefore stays positive as fresh task-signal opportunities vanish.*

842 *Proof.* The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$.
 843 A mixed group requires both a correct and an incorrect sample; the union bound gives the two bounds
 844 GP and $G(1 - P)$. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$,
 845 proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k - 1)/k^2 = 1 - 1/k,$$

846 which gives the strictly positive limiting norm for $k \geq 2$. $\square$

847 The displayed replay vector averages the fixed bank. A stochastic exemplar draw has this vector only
 848 in expectation, and a nonzero logit coordinate can be annihilated by a degenerate neural Jacobian.
 849 Replay also need not increase the next mixed-group probability: $h_G(P)$ is largest at $P = 1/2$. The
 850 claim is that a banked target can supply a separate verified signal when the fresh task gradient is
 851 unavailable.

852 A bounded advantage based on fresh samples has a different boundary behavior. The next bound
 853 includes group-dependent detached scores, and distinguishes that estimator from exact entropy
 854 regularization.

855 **Lemma J.5** (Bounded sampled scores have no probability-independent boundary force). *Condition*
 856 *on the training history, draw $A_1, \dots, A_G$ independently from p , and let a_i be any detached, possibly*
 857 *group-dependent advantages with $|a_i| \leq B$. At the on-policy point define*

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (\mathbf{e}_{A_i} - p).$$

858 *For every category b ,*

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

859 *On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages*
 860 *sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity*
 861 *$\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.*

862 *Proof.* Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{A_i = b\} - p_b| =$
 863 $2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator
 864 terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

865 The bound does not prove inevitable collapse or rule out retention by a particular bounded estimator.
 866 An unsampled mode can still change through softmax normalization or shared parameters. Exact
 867 entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero.

868 J.4 Fixed-bank replay and qualitative retention

869 Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator
 870 has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical
 871 reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (10)$$

872 This is ordinary cross-entropy against the distribution that assigns mass $1/k$ to each banked mode
 873 and zero elsewhere. It is not conditioned on the bank: decreasing (10) can raise total bank mass as
 874 well as redistribute mass within the bank.

875 In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale
 876 into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting;
 877 it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated
 878 categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P) \nabla_z P - \rho \nabla_z R_{\mathcal{B}}(z). \quad (11)$$

879 Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas J.1 and J.2, c_G has a finite
 880 positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G - 1)/G$ and MaxRL has $c_G =$
 881 $[1 - (1 - P)^{G-1}]/P$. Define

$$F_{\mathcal{B}}(z) = \rho R_{\mathcal{B}}(z) - \Psi(P(z)). \quad (12)$$

882 Then (11) is exactly gradient flow $\dot{z} = -\nabla_z F_{\mathcal{B}}$.

883 **Theorem J.6** (Verified replay prevents banked-mode extinction). *Suppose that after time T the*
884 *nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective*
885 *replay dose ρ . Under (11), define*

$$C_T = R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho}.$$

886 *Then for every $b \in \mathcal{B}$ and every $t \geq T$,*

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (13)$$

887 *Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including*
888 *asymptotically, even when the bank does not cover every correct mode. This is stronger than*
889 *finite-time positivity, which already follows from finite softmax logits without replay.*

890 *Proof.* Along the flow,

$$\frac{d}{dt} F_{\mathcal{B}}(z(t)) = -\|\nabla_z F_{\mathcal{B}}(z(t))\|_2^2 \leq 0.$$

891 Since $P(t) \leq 1$ and Ψ is increasing,

$$R_{\mathcal{B}}(z(t)) \leq R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho} = C_T.$$

892 Every $-\log p_b$ is nonnegative and their sum is $kR_{\mathcal{B}}$. Therefore $-\log p_b(t) \leq kC_T$ for every banked
893 mode, which is exactly Equation (13) for every time after T . $\square$

894 The floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical
895 $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a
896 uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the
897 experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not
898 the replay coefficient alone.

899 **Full coverage and the categorical target.**

900 **Corollary J.7** (Conditional full-coverage categorical limit). *Assume the conditions of Theorem J.6*
901 *and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Let $u_c = 1/m$ on $\mathcal{C}$. Then the*
902 *categorical mean flow converges to*

$$P(t) \longrightarrow 1, \quad q(t) \longrightarrow u.$$

903 *Consequently*

$$\min_{c \in \mathcal{C}} q_c(t) \longrightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \longrightarrow \log m,$$

904 *and conditional correct-mode collapse is impossible.*

905 *Proof.* For full coverage,

$$R_{\mathcal{C}} = \log m - \log P + \text{KL}(u||q). \quad (14)$$

906 The potential is bounded below by $-\Psi(1)$, so its descent gives $\int_T^\infty \|\nabla F_{\mathcal{C}}\|_2^2 dt < \infty$. The finite-
907 group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded.
908 Thus the Hessian of $F_{\mathcal{C}}$ and the velocity are bounded; the gradient along the trajectory is uniformly
909 continuous. Square integrability then implies $\nabla F_{\mathcal{C}} \rightarrow 0$: recurring gradients of fixed positive size
910 would otherwise contribute energy over intervals of a uniform minimum length.

911 Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

912 Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then
913 $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness
914 of this limit establish convergence of the probability vector. $\square$

915 **Positive weights and the implemented length normalization.** Uniformity of the target is separate
 916 from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w =$
 917 $-\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho_w}.$$

918 If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w||q)$, and the independent-logit limit is $P \rightarrow 1$, $q \rightarrow w$.
 919 The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate
 920 equations forces $P = 1$, then each equation forces $p_c = w_c$.

921 A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and
 922 fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$,
 923 so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the
 924 preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full
 925 bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

926 Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation
 927 then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting
 928 probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve
 929 unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-
 930 bank or stochastic neural training.

931 This distinction matters for the implemented mean-token exemplar loss. Even if each key were
 932 represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

933 Its effective categorical replay coefficient would be $\rho_w = \rho_{\text{LM}}A$. The full-coverage limit would
 934 therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual
 935 language model, $\pi_\theta(e_b | x)$ is only one contribution to the total key probability. Equal averaging
 936 of per-token exemplar scores does not imply uniform learned key probabilities; the categorical
 937 uniformity corollary is not that claim.

938 Full support is an additional premise, not a consequence of replay. In Python factors, prompts admit
 939 16–3,600 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage
 940 corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects
 941 only admitted banked modes; it supplies no probability floor for an unbanked key.

942 J.5 Complete responses and finite evaluation visibility

943 The learner scores a particular response string, while evaluation groups strings by execution key.
 944 Event containment provides a conditional bridge: one complete verified response is one way of
 945 producing its key. This bridge also makes the loss’s length factor explicit.

946 **Lemma J.8** (Retention from a joint exemplar potential). *Fix finitely many verified complete-response*
 947 *exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j | x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

948 *Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$,*
 949 *every protected exemplar satisfies*

$$p_\theta(b_j | x_j) \geq \pi_\theta(e_j | x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

950 *Proof.* The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative,
 951 so $-\log \pi_\theta(e_j | x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key
 952 b_j , which proves the stated lower bound on every protected key’s probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \Psi_x(1)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or score under another decoding law is insufficient. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K \delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero [Clopper and Pearson, 1934]. For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

J.6 Scope relative to entropy, admission, and neural optimization

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q\|u), \quad R_u(p) = \log m - \log P + \text{KL}(u\|q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$ have the unique global optimum $P = 1, q = u$ on the simplex with $P > 0$, for positive coefficients. This follows directly from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets [Pereyra et al., 2017, Agarwal et al., 2021]: replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions [Mei et al., 2020]; our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes.

998 For noisy discrete updates, a valid extension would require control of the joint loss’s upward drift,
999 optimizer bias, and accumulated error. A positive replay coefficient or small nominal learning rate
1000 does not establish these conditions. Current-gradient Adam moments, clipping, momentum, prompt
1001 scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not
1002 established the required joint-energy or update-error assumptions. The mathematical contribution
1003 is the stated categorical mechanism and conditional exemplar argument; empirical support and
1004 longitudinal exemplar scores remain separate measurements.

1005 **K Source Integrity and Reproducibility**

1006 **K.1 Admissible sources and missing checkpoints**

1007 Falcon3-1B Countdown ReplayDr.GRPO seed 59 contains two complete four-draw terminal evalu-
1008 ation sequences with identical request identities but conflicting metrics. The integrity amendment
1009 excludes that source at every checkpoint and from every efficacy endpoint, paired contrast, and
1010 aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO
1011 and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs:
1012 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial
1013 intersections are descriptive and receive no confidence interval.

1014 Trajectory records use registered sources and approved continuations or replacements, with superseded
1015 attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected
1016 checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain
1017 gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal
1018 checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares
1019 its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy
1020 and mode-count views use the same populations and gap rules. Supporting methods retain their
1021 explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the
1022 curves. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule,
1023 and source hashes.

1024 **K.2 Data, models, and scientific records**

1025 The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row
1026 identities, and construction and admission records. Its loading guide and evaluation guide identify
1027 the reported splits and preserve auxiliary data separately. The model archive links each available
1028 model–domain–method–seed export to an immutable upload revision containing original weights,
1029 configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license.
1030 The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-
1031 weighting ablation, and supporting controls to their model families; the scientific-record index and
1032 restore guide identify preserved analysis inputs and SHA-256 verification procedures.

1033 Each empirical figure has a machine-readable record identified in the figure manifest, containing
1034 its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper
1035 versions to their builders, records, and reproduction commands. Detailed source and recovery
1036 inventories remain in those records and the repository’s archived audit documents. Model availability
1037 may extend beyond the frozen analysis population; it does not change a figure’s source-admission
1038 or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories
1039 require the recorded evaluation checkpoints.

1040 **K.3 Exact continuation and validation**

1041 Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream
1042 state in Appendix D.4, in addition to model weights. State mismatch, non-finite values, missing
1043 completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures.
1044 Placement-only recovery may change hardware or wall time but must preserve the scientific update.
1045 Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints
1046 or silently incorporated from a newer archive.

1047 **L Limitations and Disclosures**

1048 The categorical theory depends on its stated geometry, energy, and coverage assumptions; it does not
1049 certify the implemented neural optimizer. Uniform correct mass requires full verified coverage, which
1050 Python’s capacity-16 bank generally precludes. The benchmark measures task-defined executable
1051 support on synthetic tasks, not free-form reasoning diversity or downstream utility. Experiments
1052 fix $G = 16$, zero reference KL, no token-entropy bonus, and one decoding protocol. The current
1053 results state every partial seed intersection and its uncertainty limits; intervals condition on the frozen
1054 prompts and evaluation requests.

1055 **L.1 AI Use Statement**

1056 Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The
1057 authors reviewed and verified the resulting content and take responsibility for the final manuscript.

1058 **L.2 Ethics Statement**

1059 The benchmark and experiment-specific data are synthetic; this study collects no human-subject or
1060 personal data. More verified modes are not inherently better: a distinct execution can be costly or
1061 risky despite passing a validator. Applications should report correctness, invalidity, and relevant costs
1062 alongside mode counts.

1063 **L.3 Reproducibility Statement**

1064 Claims use versioned results and fixed source-admission rules. Missing required results and aggregate
1065 drift cause validation to fail. Appendix B specifies split identities and evaluation settings; Appendix K
1066 links frozen scientific records, model exports, and reproduction procedures.